

植物 / 僵尸单位制作流程

本文档说明在本项目中**新增或扩展**一个可上场棋子（植物 / 僵尸）的标准流程。
与 `@requirement-workflow` 的关系：**简单换皮 / 数值微调** 可按本文直接做；**新机制、Boss、跨模块技能** 须先走需求卡片与审批（见文末）。

关联 Context： `context/modules/Chess.md`、`context/modules/Skill.md`、`context/architecture/decisions.md`

0. 流程总览



1. 开始前：判定走哪条路

类型	特征	建议流程
换皮单位	复用已有 <code>ISkill</code> 、武器、移动，只改图/数值/名称	本文 §3-§7，可跳过需求卡片
机制扩展	新被动、新索敌、新种植规则、分部件贴图	本文 + <code>docs/requirements/[单位名].md</code> （ <code>@requirement-workflow</code> 阶段 1-2）
Boss / 多阶段	多动画、召唤池、Context 键、分部件 <code>Renderer</code>	参考 <code>docs/requirements/僵王Boss行为.md</code> 、 <code>僵王分部件贴图.md</code> ， 必须 需求 + 架构分析

需求工作流（复杂单位）

`@requirement-workflow`
需求：[单位名 + 机制说明]
模块：`Chess` / `Skill` / `LevelSystem`

产出： docs/requirements/[功能名].md → 架构/影响面分析 → 审批通过后再按本文实施。

2. 策划定稿清单（动手前必填）

在 Unity 里建资源前，先明确：

项	植物	僵尸
阵营	Player	Enemy
chessName	全局唯一，与存档/图鉴/羁绊 ID 一致	同左
定位	PlantType (MainPlant / SupportPlant / ...)	波次 waveLimit、PassesWavePoolFilter
种植/站位	TileType、IPlantFunction (MainPlant / SupportPlant 等)	出场格、FindTileMethod
基础属性	血量、价格、冷却、攻速等 Property	同左
攻击	weapon 序列化类型（近战/射击/样本格等）	同左
技能	主动 ISkill + 被动 ISkillEffect 列表	同左
动画来源	单 Sprite 序列 / Reanim 多部件 / Spine,动画上添加死亡/技能/攻击事件	同左
接入点	植物商店、关卡 cards、奖励 AwardPanel	LevelData.zombieList、僵王等

3. 目录与命名规范

资源	路径
棋子 Prefab	Assets/Prefab/ChessPrefab/[分类文件夹]/[chessName]/[chessName].prefab
PropertyCreator (植物)	Assets/Resources/ChessData/Player/[chessName].asset
PropertyCreator (僵尸)	Assets/Resources/ChessData/Enemy/[chessName].asset
技能脚本	Assets/Script/Skill/ISkillEffect_Plant/ 或 ISkillEffect_Zombie/、ISkillPassive/
动画	Prefab 同目录下 动画/ 子文件夹，或 Assets/Reanim2UnityAnim/Output/
Shader 材质	默认 Assets/Shader/BaseShader/BaseMaterial.mat（受击闪烁等）

注意：脚本目录为 `Assets/Script/`（单数），不是 `Scripts`。

4. 美术资源

4.1 简单单位（单 Sprite + Animator）

1. 准备 `idle / attack / skill / death` 等序列帧或 Animation Clip。
2. 图标 `chessSprite`：用于商店、图鉴、种子包 UI。
3. 可选：种植预览图、卡牌 Prefab 用图。

4.2 PVZ Reanim 单位（多部件僵尸/植物）

1. 将 `.reanim` 与贴图放入 `Assets/Reanim2UnityAnim/reanim_all/`（或沿用已有资源，见 `docs/Reanim2UnityAnim-inventory.md`）。
2. 菜单 `Window → Reanim2UnityAnim`：
 - 选择 `.reanim`
 - 配置 `center`、分段 `customPartitions`（`idle`、`walk`、`attack`、`death` 等）
 - 点击 **生成** → 输出到 `Assets/Reanim2UnityAnim/Output/[单位名]/`
3. 生成结果挂到 Prefab 子层级（多 `SpriteRenderer` / 骨骼名如 `arm`、`head`）。
4. 分部件换皮（如僵王）：在自定义 `AnimatorController_*` 上配置多组 `Sprite`，见 `docs/requirements/僵王分部件贴图.md`。

4.3 特效与音效

- 技能特效：优先 **预制体 + 对象池** 或粒子 Prefab 放在 `Assets/Resources/Effect/`。
- 音效：经 `AudioManage` / 棋子 `AudioPlayer`（按现有单位引用方式）。

5. 创建数据与 Prefab

方式 A：编辑器向导（推荐新单位骨架）

菜单 `Tools → ChessEditorWindow`：

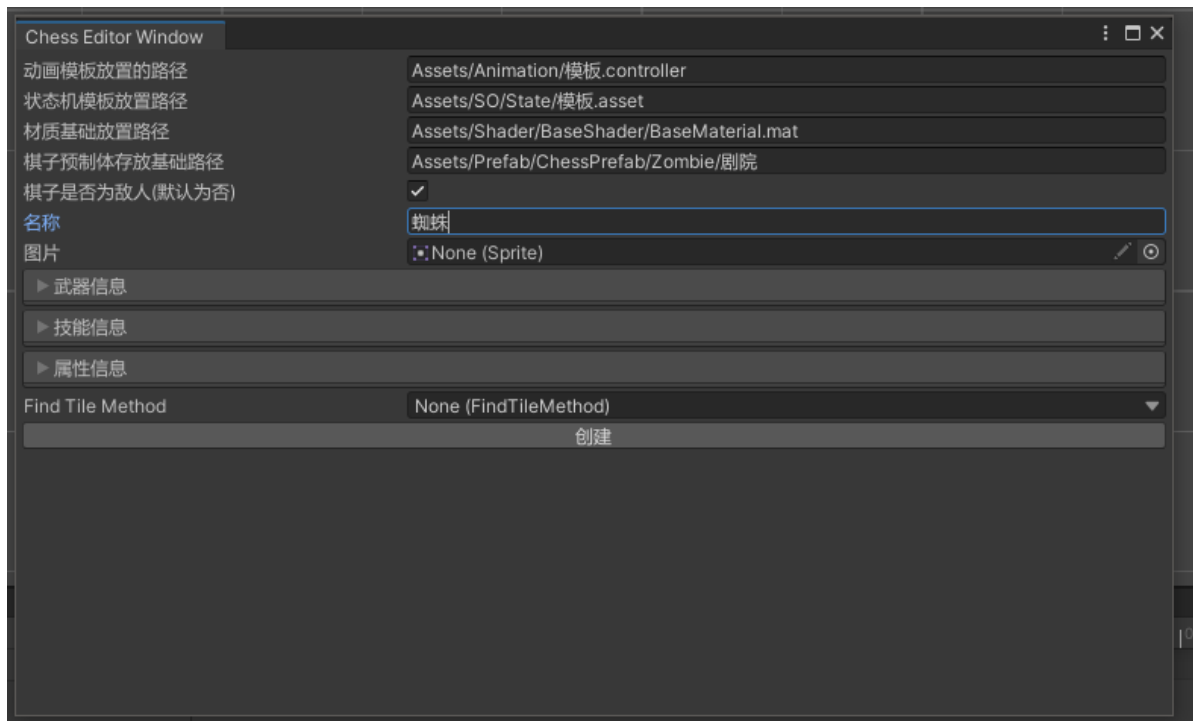
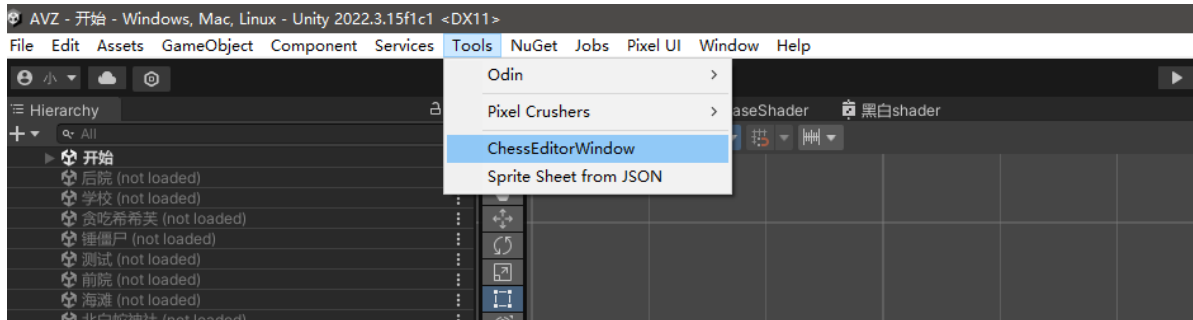
字段	说明
<code>enemy</code>	僵尸勾选，植物不勾
<code>chessName</code>	与后续 SO、Prefab 名一致
<code>chessSprite</code>	列表图标
<code>weaponData</code>	<code>[SerializeReference]</code> <code>weapon</code>
<code>skillData</code>	主动/被动 <code>ISkillEffect</code> （需在窗口外建好类型）
<code>propertyData</code>	血量、价格、 <code>PlantType</code> 、 <code>TileType</code> 、 <code>IPlantFunction</code>
<code>findTileMethod</code>	僵尸移动/寻路（植物可为空或种植逻辑）

点击 **创建** 后自动生成：

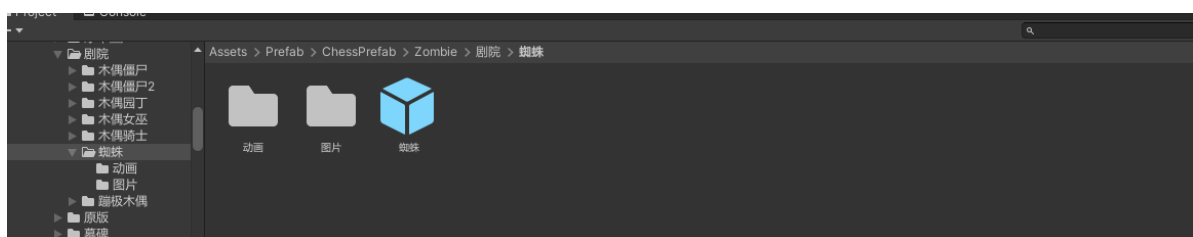
- Assets/Prefab/ChessPrefab/[chessName]/ (含 动画/、 图片/ 子目录)
- [chessName].prefab (含 Chess、 BoxCollider2D、 Animator、 AnimatorController、 sprite 子物体)
- Assets/Resources/ChessData/Player|Enemy/[chessName].asset (PropertyCreator)

创建后必做：在 Inspector 中把 **Animator Override Controller**、**StateGraph** (Assets/SO/State/模板.asset) 补全绑定；向导仅搭骨架。

截图步骤参考：



主要就是填这个表，只有基础路径，是否敌人和名称是需要填写的，后面的那几个东西图片啊 武器信息啊什么的我不建议在这搞，到时候直接去对应的prefab下面填写就好了。路径就是放在哪个文件夹。名字就是叫什么了，是否为敌人是决定这个单位的PropertyCreator放在哪。填完之后点击生成就会在对应的路径下生成<名称> 的文件夹。里面有一个chess预制体和一个动画文件夹，一个图片文件夹：其实在对应的Resources/ChessPre/Enemy (Player) 下面还生成了一个对应PropertyCreator



方式 B：复制现有单位（推荐机制相近时）

1. 复制最接近的 Prefab 文件夹与 `PropertyCreator.asset`。
2. 重命名 `chessName`，全局替换引用。
3. 修改 `PropertyCreator` 与 Prefab 上 `SkillController` / `AttackController` 的序列化字段。

PropertyCreator 核心字段

```
chessName / chessSprite / chessDescription / chesseEffect
baseProperty (Property: hp、price、cd、waveLimit...)
chessPre → 指向本单位 Chess Prefab
plantType / chessTileType / plantFunction (植物)
plantTags / fetterMemberId (羁绊)
PlantCardPre / PlantEntrepotCardPre (植物商店 UI，可选)
```

`PropertyCreator.chessPre` 必须指向带 `Chess` 组件的 Prefab，否则 `CreateChess` 无法实例化。

6. Prefab 组件检查表

根物体 `Chess` 上序列化模块（与现有单位保持一致）：

模块	职责
<code>PropertyController</code>	<code>creator</code> → 本单位 <code>PropertyCreator</code>
<code>SkillController</code>	<code>activeSkill</code> / <code>passiveSkill</code> ([SerializeReference])
<code>AttackController</code>	<code>weapon</code>
<code>MoveController</code>	<code>tileMethod</code> (<code>FindTileMethod</code>)
<code>StateController</code>	<code>stateGraph</code> (Idle/Attack/Skill/Death)
<code>BuffController</code>	默认即可
<code>AnimatorController</code> 子类	绑定 <code>sprite</code> 、 <code>animator</code> ；特殊僵尸可继承 <code>AnimatorController_ZombieKing</code> 等

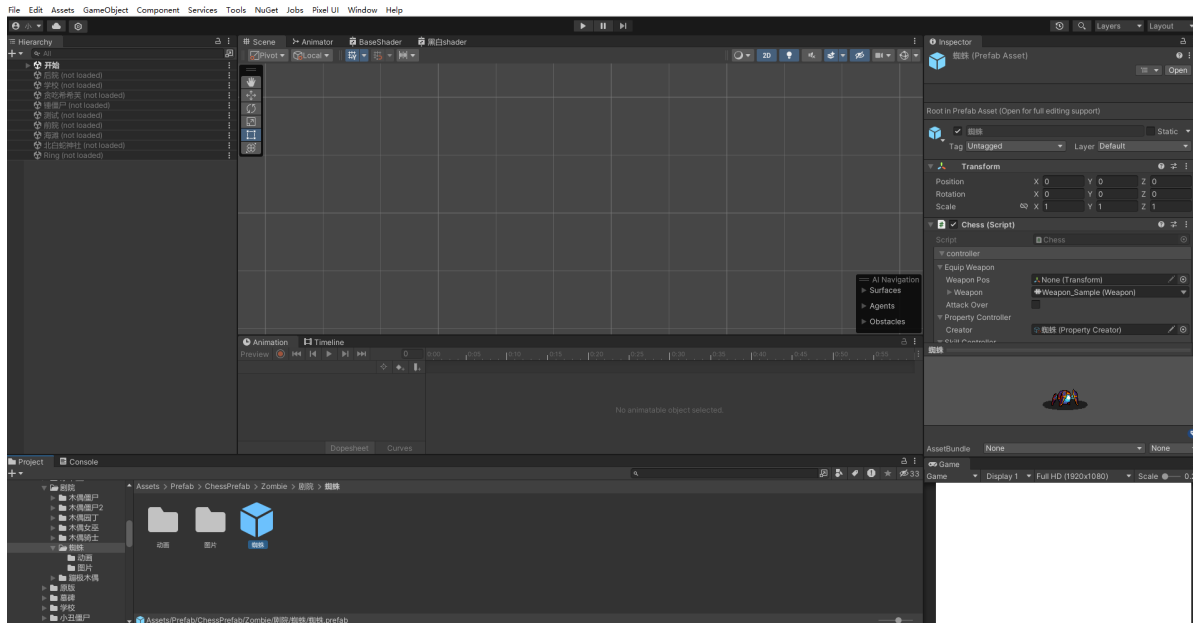
技能实现约束（必读 `.cursor/rules/unity-skill-no-runtime-monobehaviour.mdc`）：

- 禁止为技能 `AddComponent` 运行时专用 `MonoBehaviour`。
- 使用 `ISkill` / `ISkillEffect` + `WhenEnterGame` / `OnRemove` / `SkillContext` / 协程 `user.StartCoroutine`。
- 参考：`PassiveSkillEffect_MinerZombie`、`ISkillEffect_BigRabbit`。

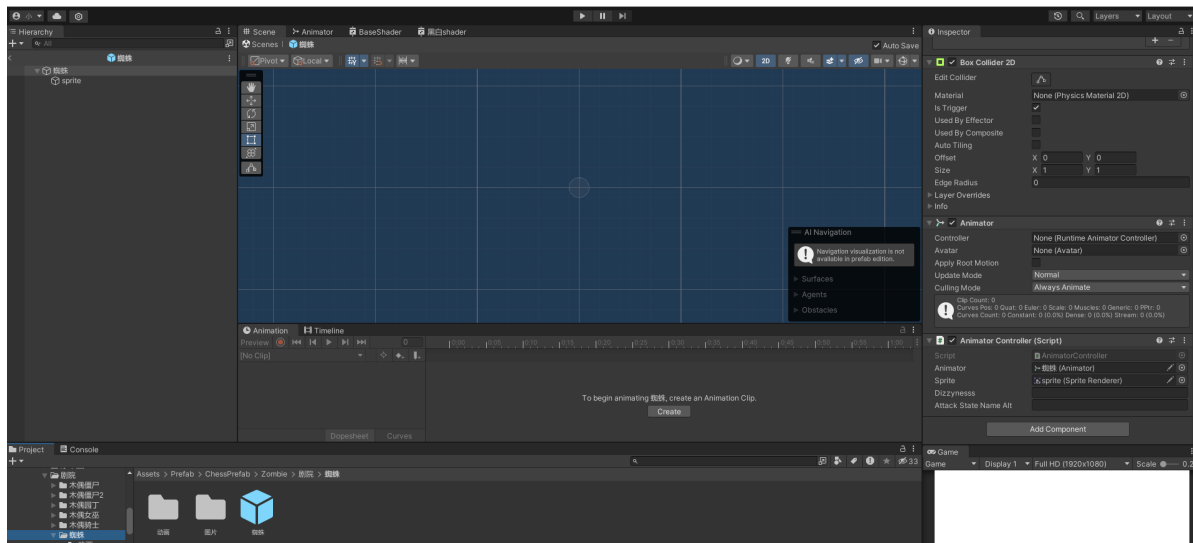
动画事件（在 Animation Clip 上）：

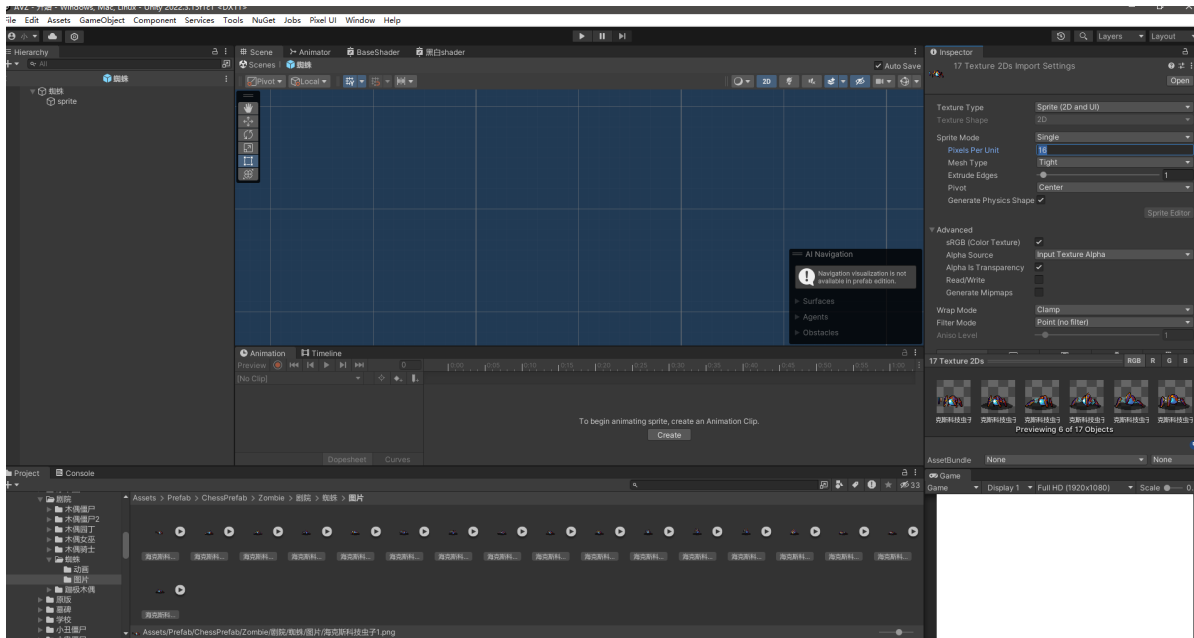
- 普攻伤害：`Chess.TakeDamage()`
- 技能生效：`Chess.UseSkill()` → `SkillController.UseSkill()`
- 死亡收尾：状态机进 `DeathState` 后 `Chess.Death()`（回收）

然后就到最关键的Prefab配置了：



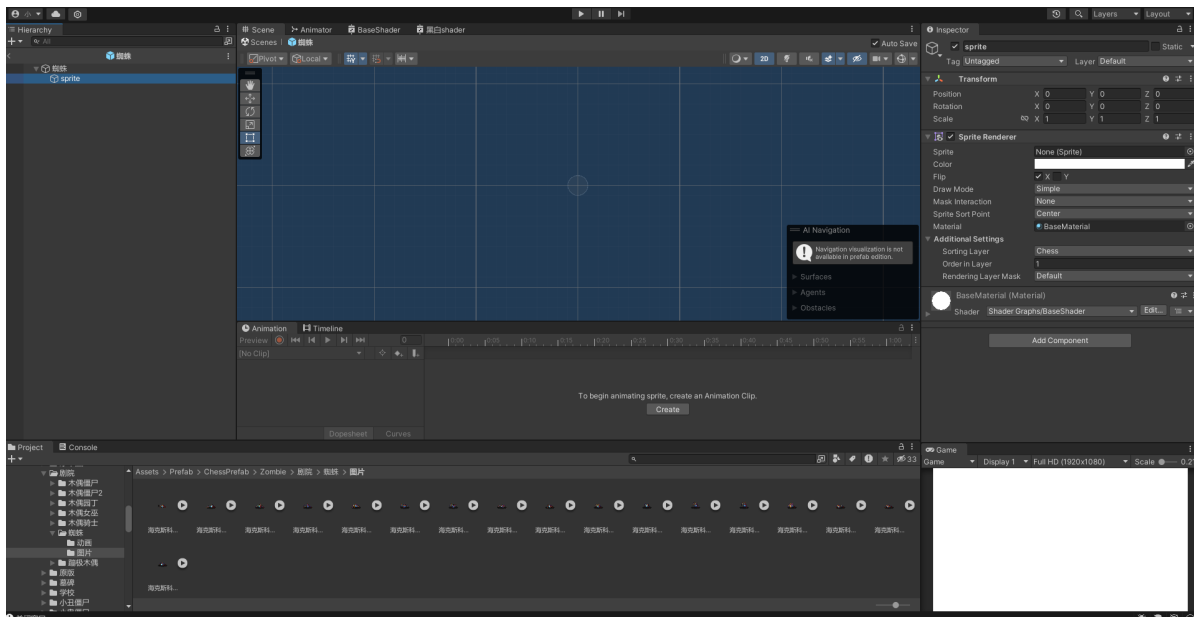
点击prefab文件 在Inspector窗口点击Open 进入预制体编辑场景





把图片素材都拖入图片文件夹。点击第一张图片后按shift再点击最后一张图片全选所有图片。在Inspector面板将TextureType设置为Sprite(默认), SpriteMode设置为Single(Multy的场合以后再教, 意思是这一张图就是一帧 而不是多帧的图), PixelPerUnit设置为16, 意思是这张图的像素大小, 只要知道所有图片素材保持一致就好。FilterMode设置为Point (意思是像素画) 其他东西都设置为默认。(其实有需求的话是要改Pivot, 不过我们可以修改sprite位置 所以不用手动调整Pivot) 最后点击Apply就行。这样就算配置好图片素材了

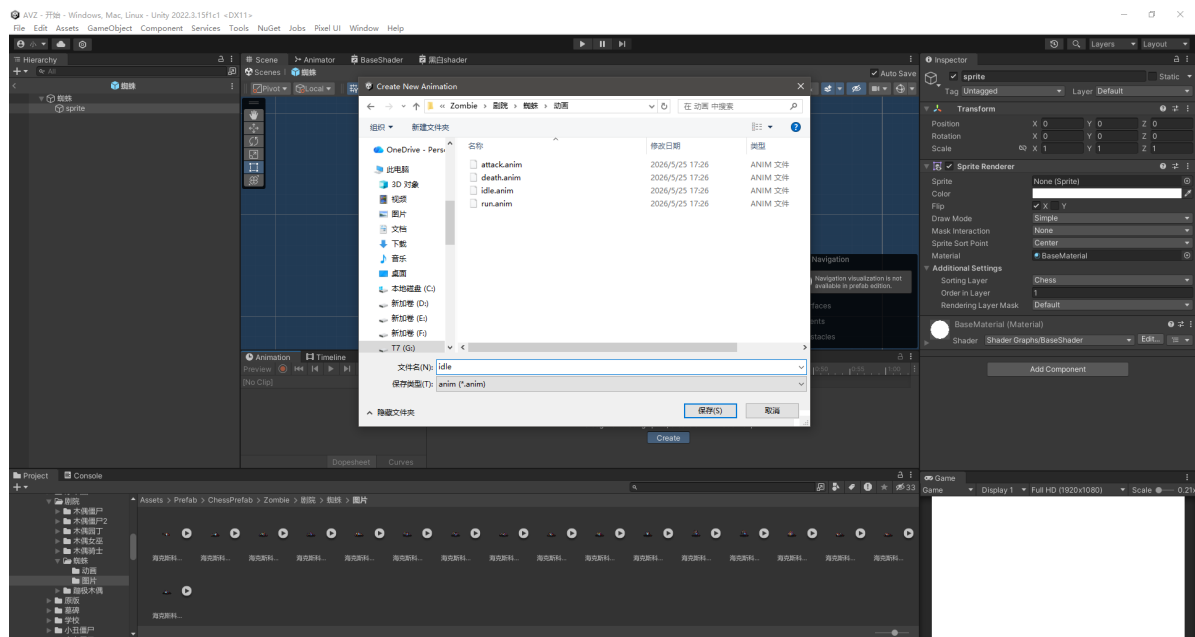
下一步做动画



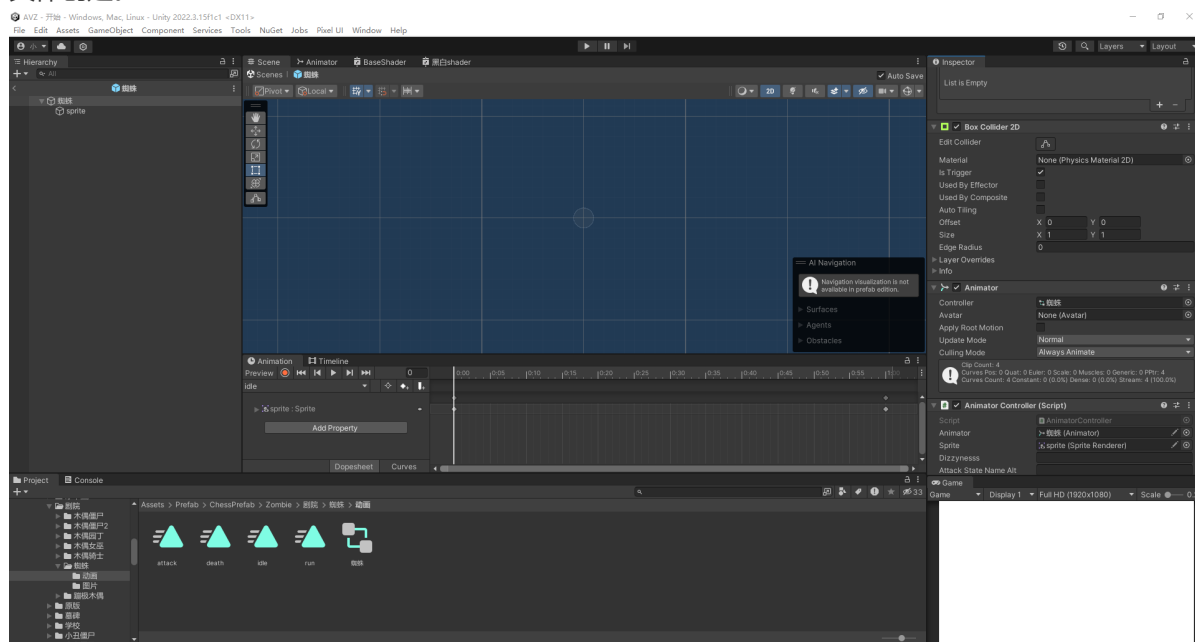
点击sprite在Inspector面板 有个SpriteRenderer组件。这个组件就是显示这个单位长什么样子组件。sprite就是要显示的图片样式, 我们把Project里的单张图片拖入, 那么这个单位的默认长相就是对应的图片了。帧动画的原理就是每间隔1/12秒修改一次spriteRenderer的sprite, 图片的切换就形成了动画。Color是图片颜色, 因为我们图片是上好色的所以默认是白色就好; Flip是朝向, 因为我们默认所有图片朝向是向右, 然后如果你的角色画的朝向是向左的, 那你就点一下这个Flip。同理, 因为僵尸图片默认也是向右的, 但他在生成的时候会被y轴旋转至朝左就不需要改Flip。但是如果图片是朝左的就需要点这个Flip。反正图片是朝左的就点Flip。

AdditionalSetting里面只有Sortinglayer和OrderInLayer需要考虑。Sortinglayer表示渲染层级，层级越高渲染优先级越高。同层级下OrderInLayer越高 渲染顺序越高。这个我在代码里有设置 所以不用手动修改。

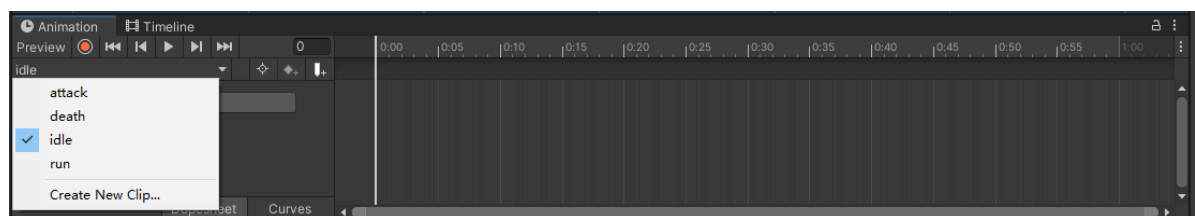
在做好一帧后，点击Animation里面的Create按钮来创建动画。（不知道Animation窗口在哪的话 点击上方栏Window->Animation->Animation打开这个窗口）



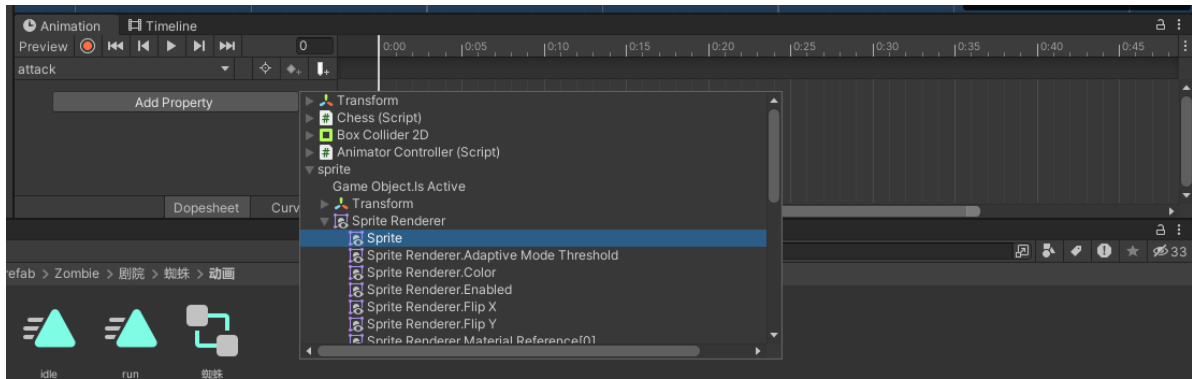
如图示，一半要创建(僵尸: idle run attack skill death) (植物: idle attack skill death)有些植物/僵尸可能没有skill(比如普通僵尸，豌豆射手)，有些可能没有attack(比如向日葵，旁观者僵尸)。具体情况具体创建。



然后你就能看到在动画文件夹下有四个动画和对应的AnimatorController,也就是动画状态机的控制器。把这个控制器放到Chess(蜘蛛)的Inspector的Animator下面。然后在animation窗口就可以开始动画配置了。

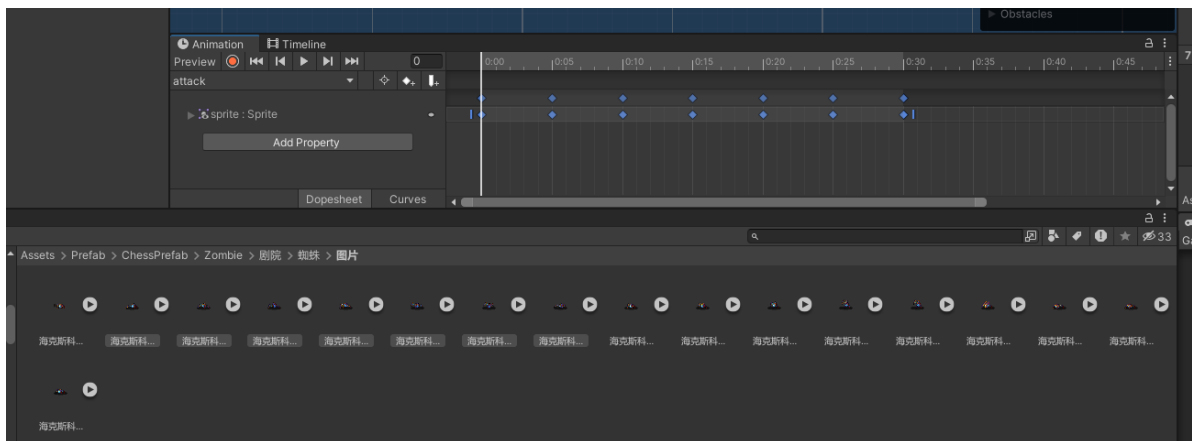


点击右上角<动画名>下拉可以看到要修改的动画，带年纪CreateNewClip可以创建新的动画，这个动画会被自动加入到动画状态机（Animator）里面。我们选择attack做为示例

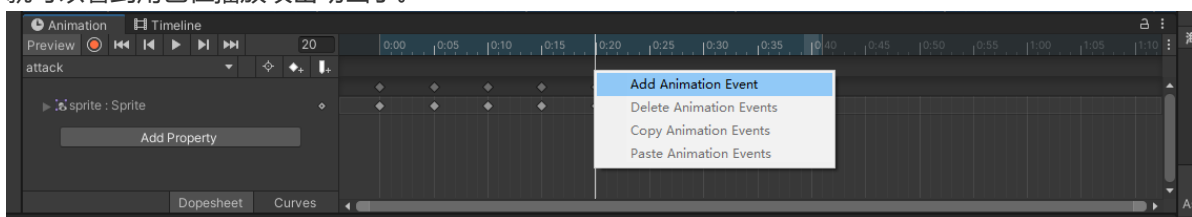


切换到attack动画后，点击addProperty 意思是这个动画时间轴里面你有哪些属性是需要随时间改变的。因为我们的动画是挂载在Chess(蜘蛛).prefab下面的 而我们的图片（spriteRender组件）是放在sprite子物体上的。所以我们要添加sprite->SpriteRender->Sprite。按shift+滚轮可以拉到右边 点击加号就可以把对应属性放到时间轴上。

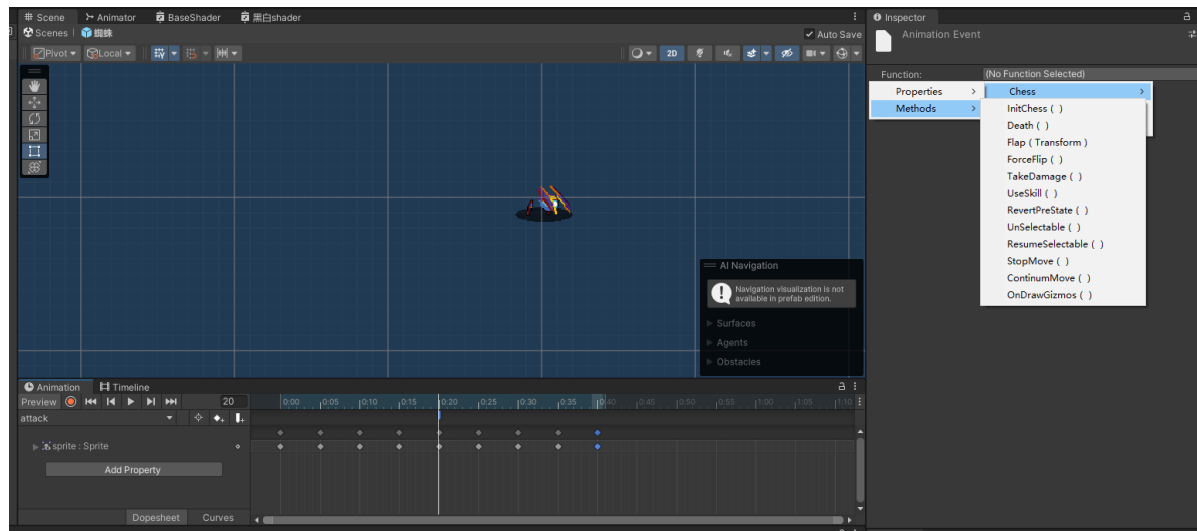
这里解释一下为什么不把SpriteRender直接放在chess上，因为我动画和角色行为是分离的。比如说一个僵尸他的动画是大跳砸地，如果他的动画也放在角色本体上，那他大跳的时候可能就到其他行了，就会被其他行的植物打到，但他实际的判定位置应该是要在原本行的。同理还有我想让这个角色在生成的时候他的脚能站在对应的草地的中心，或者说我希望同行的所有植物都站在统一一条线上。那么他们本体的chess的transform.position.y应该是一样的。但是你图片有大有小 有正有歪，你可能实际放进去发现设置y为零的时候脑袋在草坪中心。这是不符合视觉效果。那如果是分离的情况 就可以直接修改子物体的sprite的位置 让他对准，就不会影响chess的判断逻辑的情况下，让视觉逻辑也正常，还有很多其他好处就不一一说明了。**重点就是要知道动画改变的是(子物体sprite)的(spriteRender(组件)的sprte (组件对象))**



然后就是把图片多选（shift+点击），然后拖动到animation窗口中。然后调整每个节点的位置（shift+点击节点可以整体移动所有节点），一秒对应12帧，在这里就是每0.05放一帧。点击播放按钮就可以看到角色在播放攻击动画了。



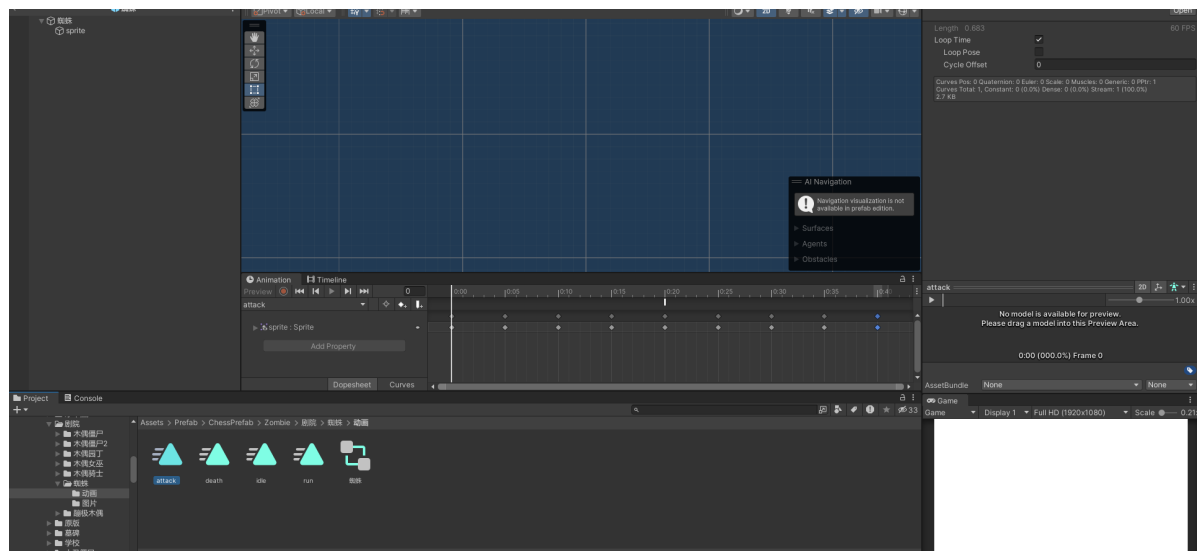
点击播放键左右的逐帧播放按钮 调整到准备造成伤害的一帧（远程攻击的话就是发射子弹的那一帧），在对应位置右键，点击add Animation Event 添加角色攻击事件



然后点击事件的那个小按钮，在Inspector窗口 点击Function：选择Chess->Methods->TakeDamages（） 添加攻击事件。这个函数是我放在Chess里面 用于调用attackController攻击事件的函数。这里不用深究 知道动画播放到这里的时候会触发攻击事件（比如直接造成伤害，又或者是发射子弹）就可以了。

用相同的方法创建idle动画（无需绑定事件） skill事件（需要绑定UseSkill事件） death动画（需要绑定Death()事件一般是动画的最后一帧） run动画（无需绑定事件，因为移动原理我是放在状态机里移动的不需要动画驱动）。以及其他特殊动画 具体情况具体创建。

设置完帧动画后再管理一下动画的循环关系：

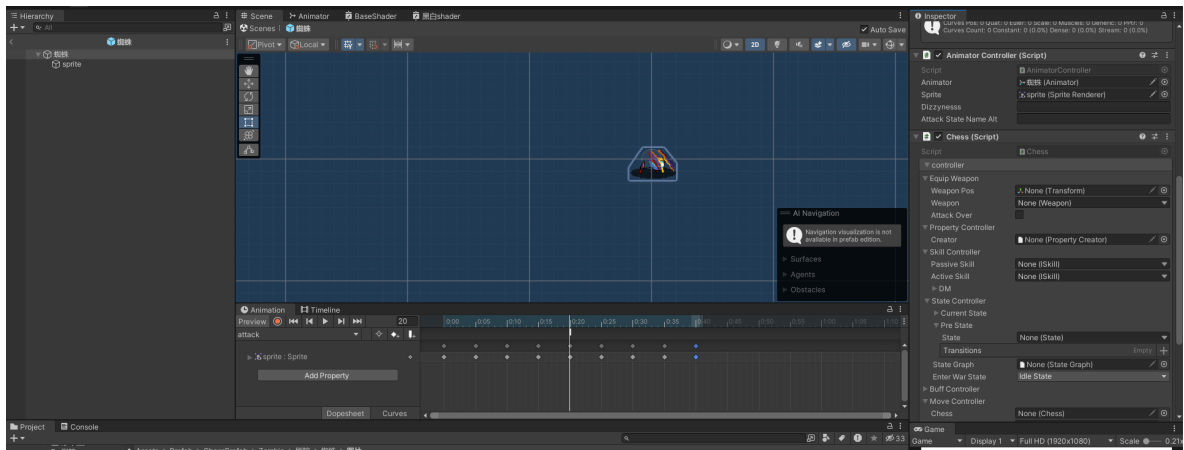


点击动画 在inspector窗口上有个looptime。勾选可以让这个动画进入循环播放。一般idle， attack， run会勾选这个； skill和death动画等不会勾选

在做好所有动画后进入下一步

角色配置:

再次点击chess（蜘蛛）， 在Inspector窗口找到Chess组件

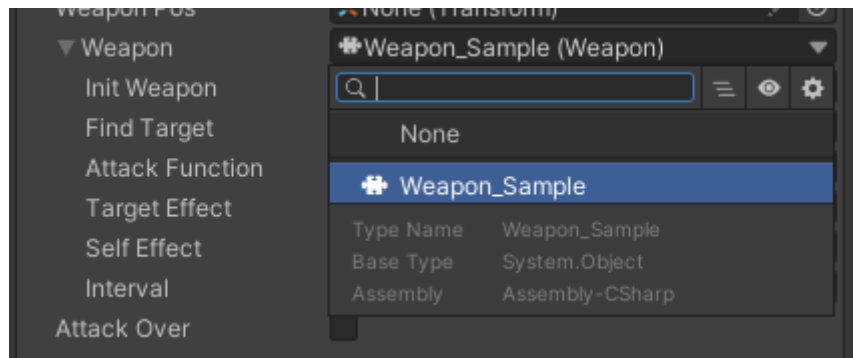


点击controller

首先我们配置EquipWeapon:也就是角色的攻击系统:

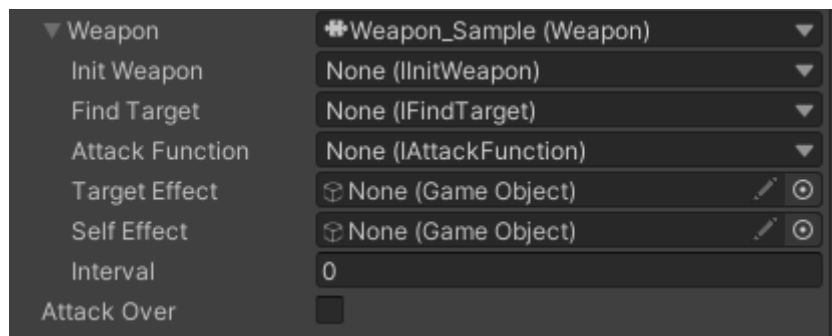
1.WeaponPos(次要):这个主要是远程单位要设置, 指子弹射出的位置, 因为不同动画的射出口不一样, 所以配置的位置也不一样, 所以特地留了一个对象用来配置这个。一般就是在chess下面创建一个空物体shootpos, 然后移动位置到射击口, 然后再把shootPos放到WeaponPos里面。

2.Weapon:



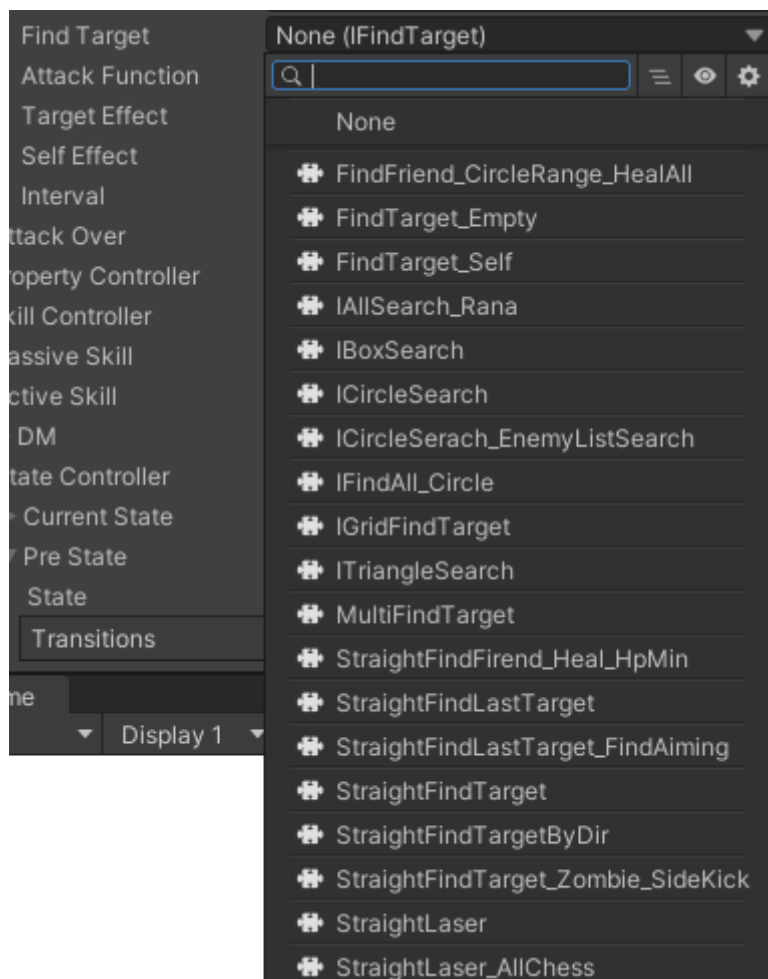
点击weapon设置为weapon_Sample: 这个是常规的攻击方式 所有的植物僵尸都可以用这个。主要是我之前是打算做成可以换武器的或者有特殊攻击的, 所以weapon做成了接口的形式可以继承。不过目前发现只要Weapon_Sample就够用了, 就没有新的。

配置Weapon_Sample (重要) :



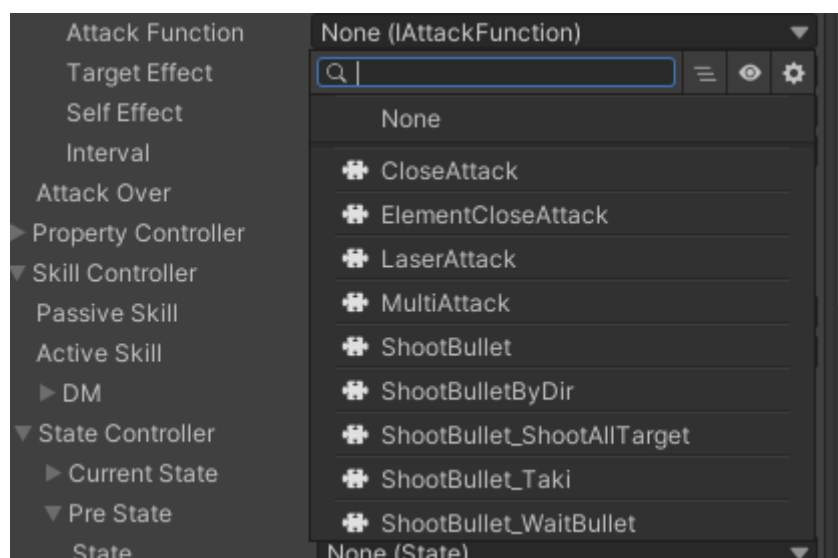
InitWeapon:武器初始化, 这个暂时没有使用啊无视就好。如果以后有什么初始化的话, 额 比如说有什么武器是用一段时间升级 然后要复原什么的可以考虑在这里写。一般情况用不到

FindTarget:武器的寻敌方式:

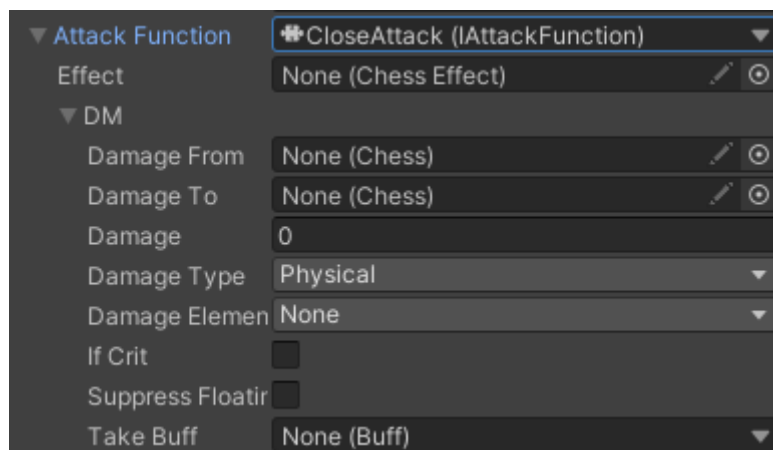


可以看到有很多种啊，因为有很多需要适配的武器，比较常用的是StraightLaser和IGridFindTarget;前者是根据Property中 AttackRange的距离来判断前方的单位是否在自己的攻击范围；后者是类似明日方舟那种按格子来配置攻击范围。一般的僵尸用前者就可以了。

AttackFuntion:武器的攻击方式



就是chess是怎么攻击敌人的。最常用的是CloseAttack和ShootBullet；CloseAttack就是直接造成伤害；ShootBullet就是发射子弹。一般的僵尸都是CloseAttack

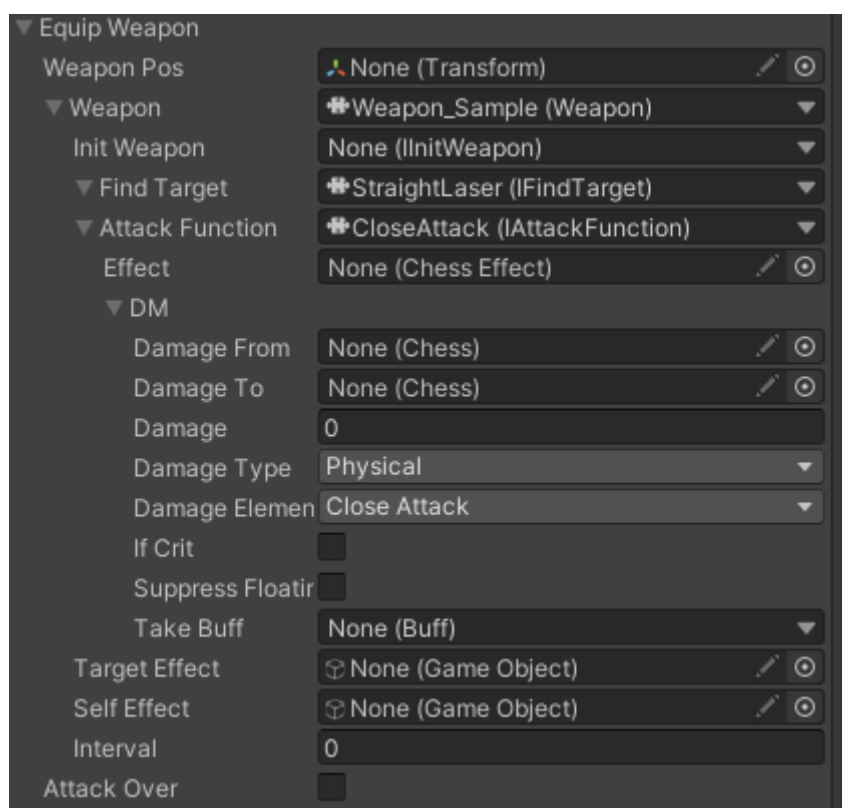


设置为CloseAttack后，Effect是不用配置的,这个是某些攻击会生成一些特效的时候我留的接口，一般用不上；然后DM主要是设置DamageType和DamageElement 一个是攻击类型，就是物理魔法真伤这种；另一个是攻击属性，就是伤害是爆炸（无视防御），切割（会切割绳子），穿刺（会击破气球），碾压（如果目标体型小于自身则秒杀对方）

TargetEffect.SelftEffect：也是生成特效的，只不过这个特效一个是生成在自身位置，一个是生成在地方位置。

interval：攻击间隔：一般是用在攻击动画帧数小于实际攻击间隔的时候使用。一般来说攻击动画attack直接设置为loop就可以了。但是有时候我想让他的攻击间隔很长，但是我动画可能就画了8帧，那我可以让attack播完后先播放idle动画。然后经过interval时间后他会再次播放attack动画。一般情况不设置就行。

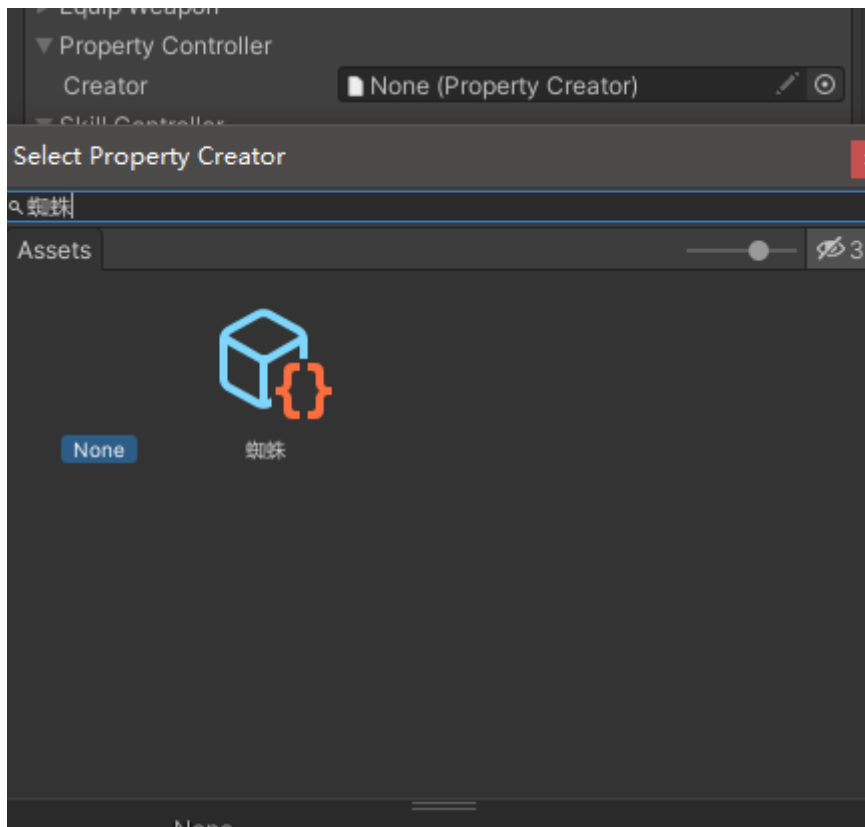
AttackOver:这个是内部逻辑 不用管



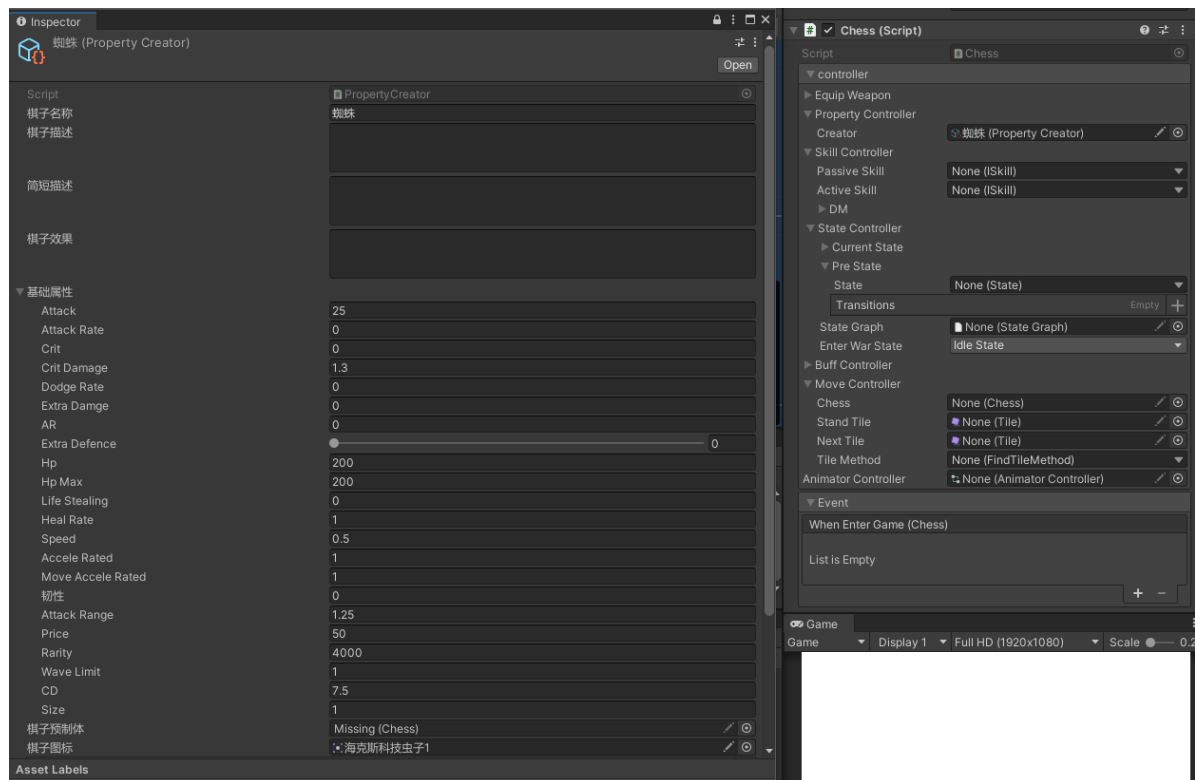
最后配置完差不多是这样。

配置PropertyController:

这个就是属性配置了 其实就一个选项Creator 但是我们可以在这里配置具体的属性 所以也放在这里说了：



点击那个小圆圈，然后会弹出一个选择界面。直接搜索chess同名，然后双击就可以。



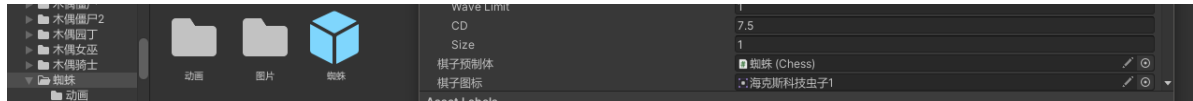
选中后 直接点击小圆圈旁边的那个笔✎图标就可以打开属性配置界面了；前面就是介绍什么的，主要是配置基础属性。最重要的几个属性是**Attack**（攻击力）**HP**和**HPMax**（生命值）**Speed**(移动速度，僵尸要配置) **AttackRange**(攻击范围，一格是2.5 僵尸一般半格1.25)

Price(阳光费用。这个植物和僵尸都要设置，植物的话就是阳光花费，僵尸的话就代表他一波能出多少。一个普通僵尸的价值是25)

Rairty(稀有度，主要是僵尸，代表每波出现的概率，计算方式是 这只僵尸出现概率=（僵尸Rairty）/(本波所有僵尸出现概率之和))

CD: (植物的冷却时间) **size**(体型, 一般为1, 食人花是5, 巨人僵尸是10, 冰车是4)

棋子预制体: 系统使用PropertyCreator创建的时候需要这个, 直接把对应的Prefab拉进去就可以了



棋子图标: 主要是种植栏显示的UI图, 从图片里拉一张就好

羁绊列表: 植物或者僵尸拥有的羁绊。比如所属乐队 (结束乐队), 又比如使用的乐器 (吉他)。

羁绊成员ID: 同ID的单位计算羁绊时候只算一次: 比如波奇和吉他英雄的羁绊ID都是后藤一里, 所以就算你选了吉他英雄和波奇也只会让结束乐队羁绊+1;

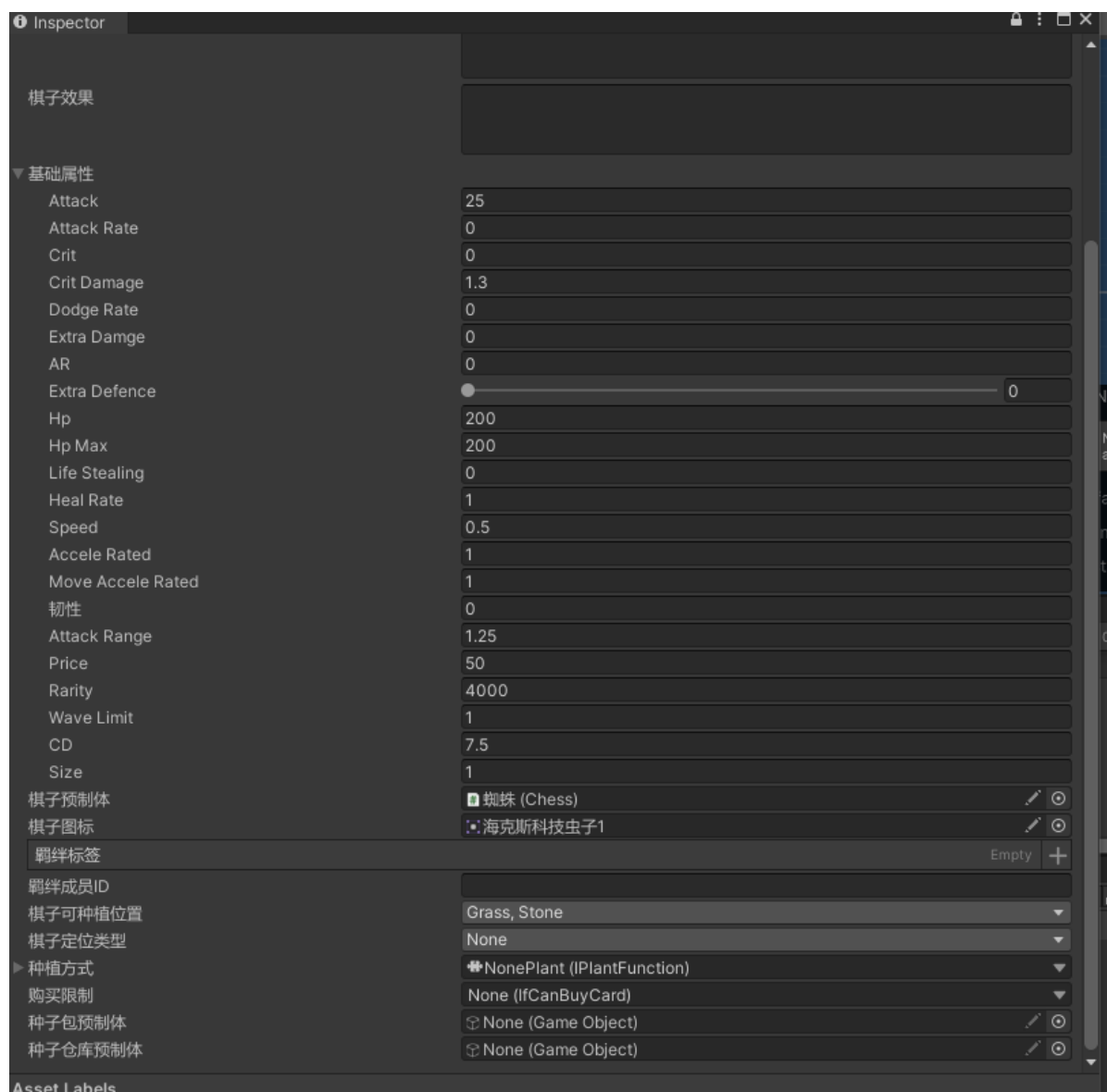
棋子可种植位置: 棋子可以种植的地形, 一般植物只能种植在Grass草地上。一般僵尸只能种植在Grass和Stone上。

棋子定位类型: 主要是植物有这个区别 这个比较复杂, 一般的植物都是MainPlant 视为主植物, 一个草地格只能有一个主植物。SupportPlant辅助植物是可以种植在主植物上的植物, 比如南瓜罩。一个主植物只能拥有一个辅助植物; PotPlant花盆植物, 睡莲花盆等可以改变地形的植物。一个草地格只能拥有一个花盆植物; Consume: 消耗品 如咖啡豆, 可以直接种植在植物上有特殊效果, 使用完就消失; LimitType: 有特殊限制的植物 (比如可种植数量, 场上必须有某些单位等)。僵尸一般选None就行

种植方式: 根据不同的定位类型选择对应的种植方式。一般植物选择MainPlant; 然后还有升级植物必须种植在基础植物上; limit植物只能种植一个等; 僵尸的话一般选NonePlant;

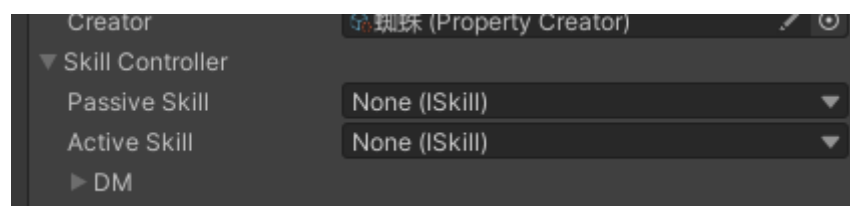
购买限制: 能否购买的判断标准, 比如说有些植物只能种植一个, 有些植物只能种植在基础植物上升级。一般选None就可以

种子包预制体和种子仓库预制体: 就是颜色不一样的框框, 升级植物是紫卡, 消耗植物是白卡。一般不管



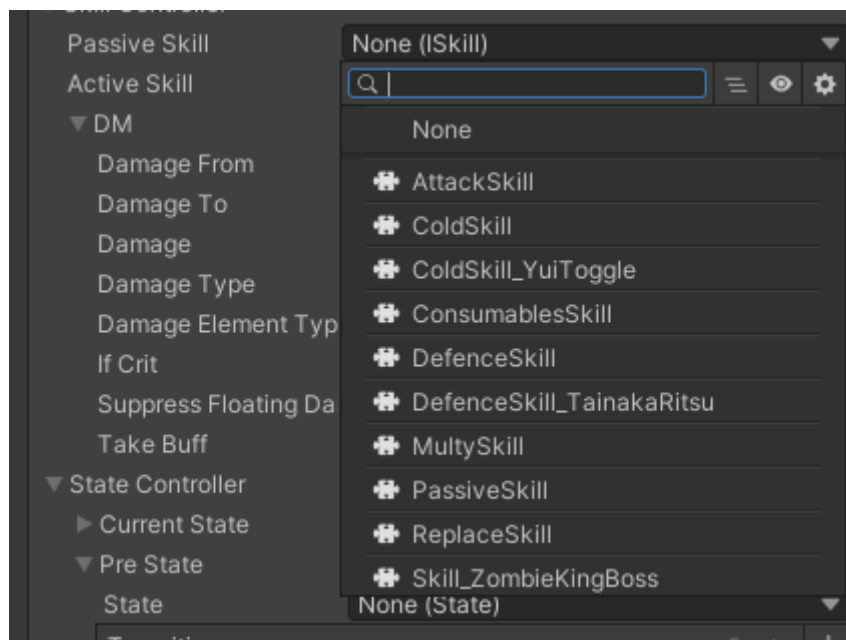
填完差不多这样

SkillController:技能控制系统

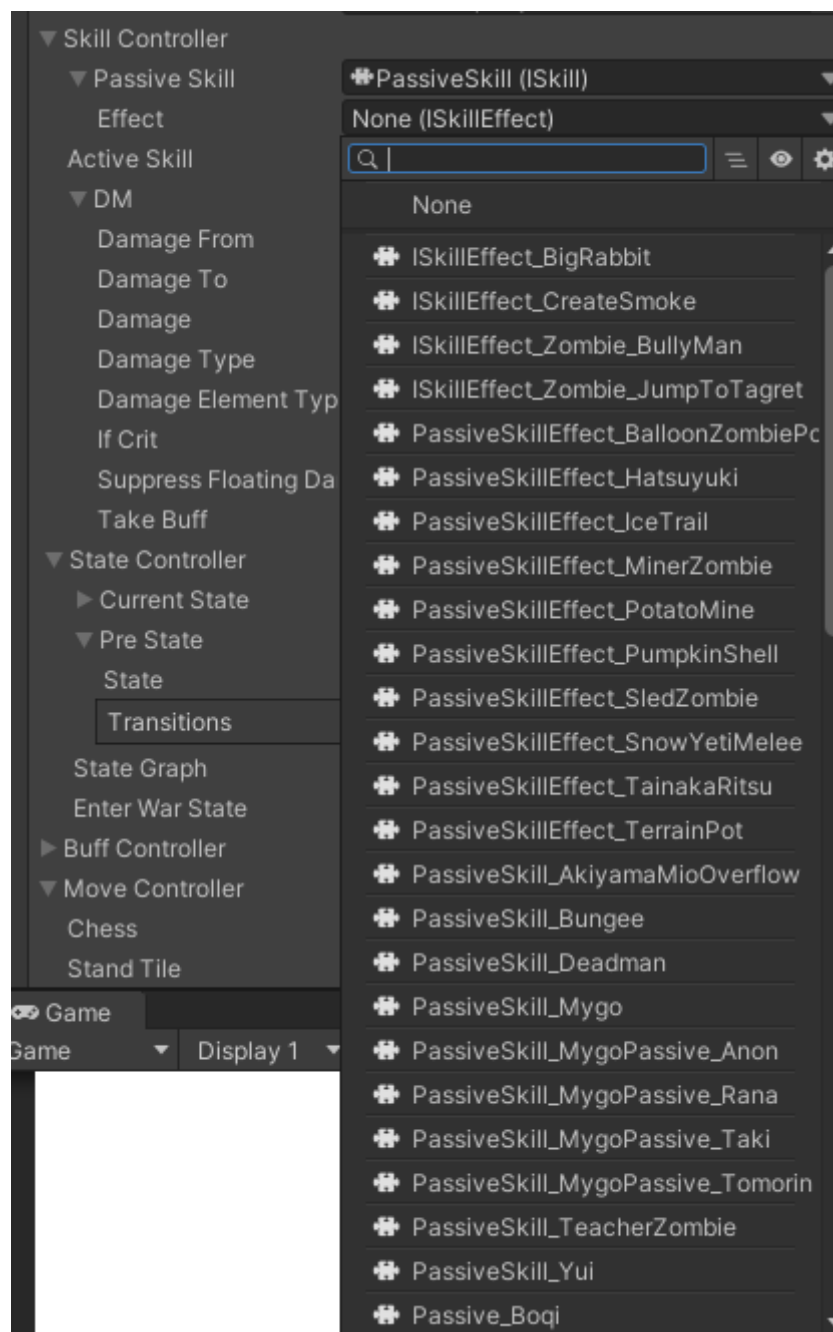


这个也是比较重要又痕迹吧复杂的系统

首先是被动技能：



选择passiveSkill

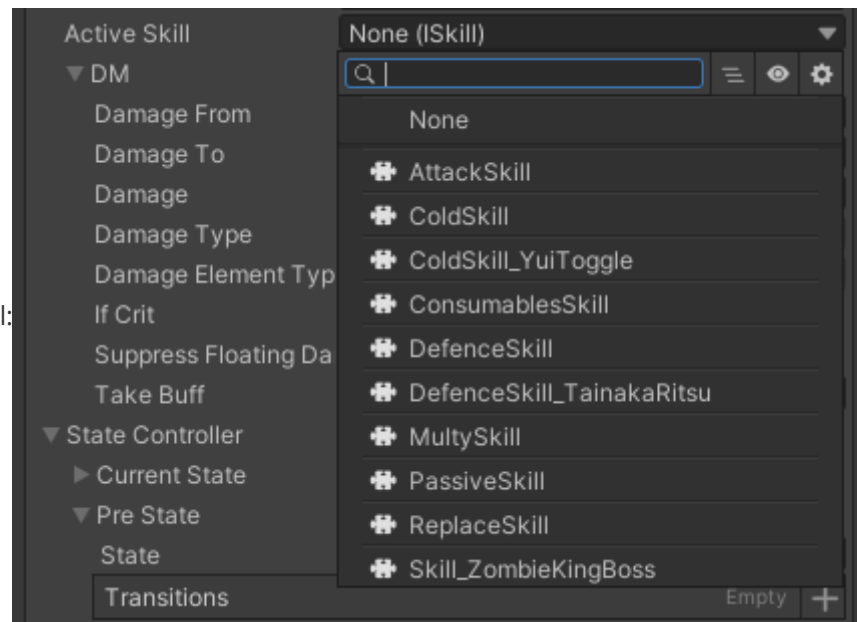


然后点击Effect选择对应的被动技能效果。

被动技能没有触发条件，在植物或者僵尸入场的时候会调用一次被动技能效果。如果没有被动技能就选None就行

然后是主动技能：

点击activeSkill:



不同类型的技能有自己的判断方式

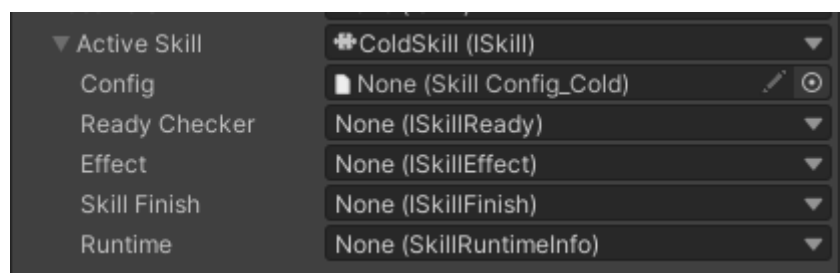
AttackSkill:是按攻击次数来判断是否冷却完毕（攻击n次后可以使用的技能）

coldSkill:冷却完毕就可以使用技能

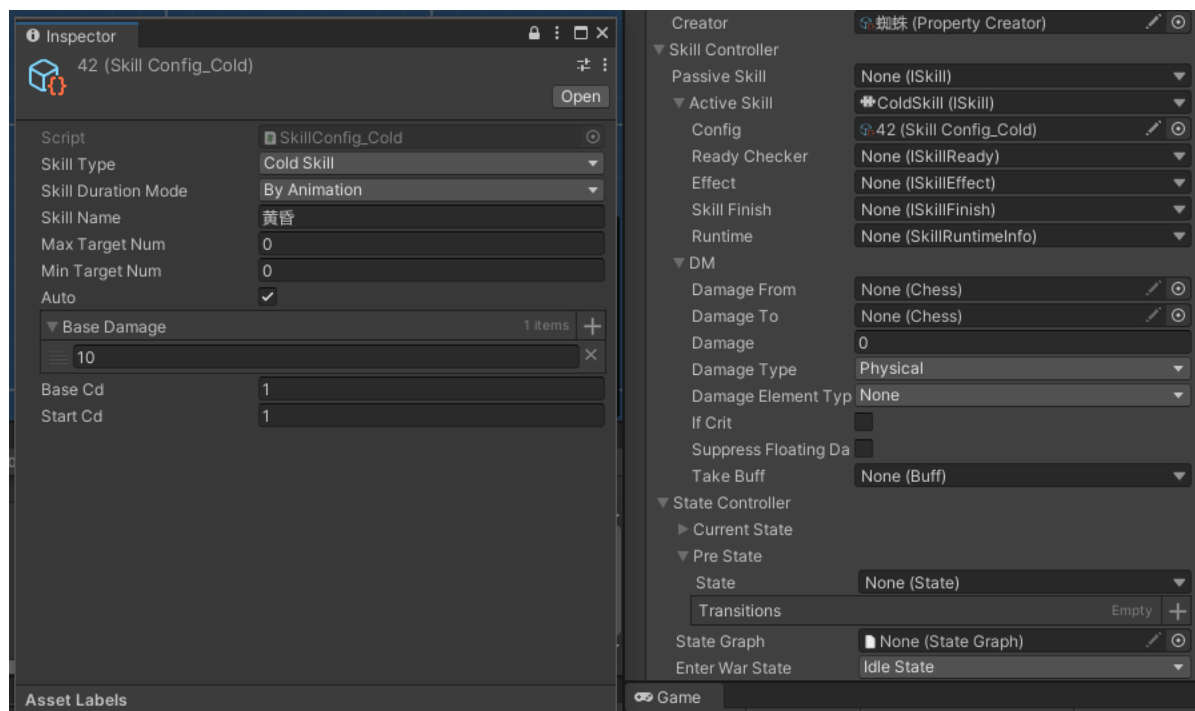
DefenceSkill：被攻击n次后可以使用的技能

一般来说这三个就够用了。

这里选ColdSkill作为示例：

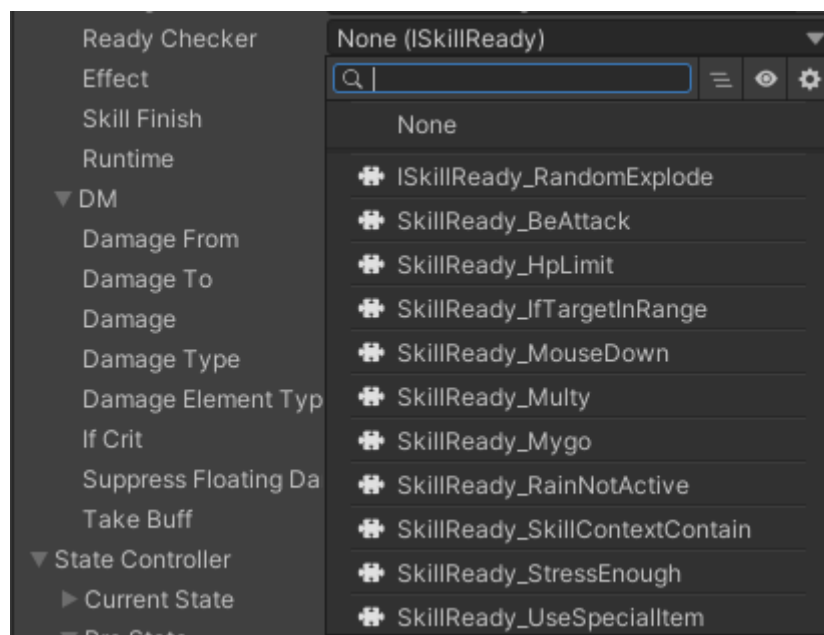


config是这个技能的一些基础属性

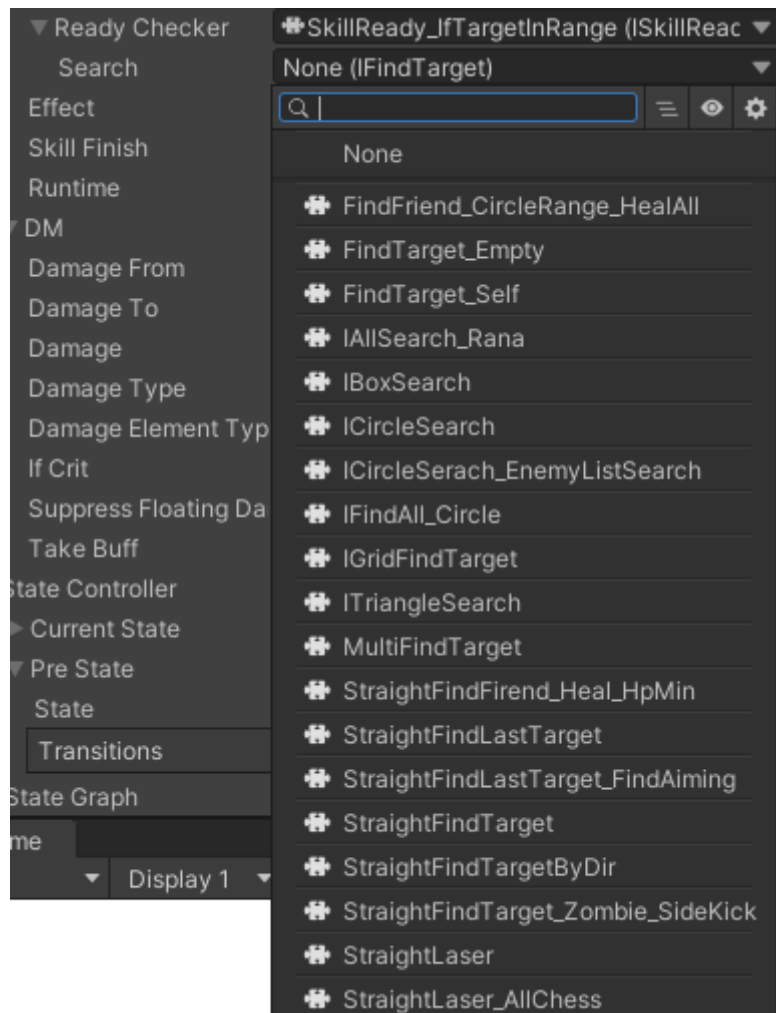


选择后可以看到需要配置的信息：主要是BaseCD和StartCD,用于控制开始冷却和最大冷却。比如你BaseCD是10，StartCD是5说明第一次用技能5s后就能使用，之后的每次技能就都是10s了。Max,Mintarget 代表这个技能可以选中的最大，最小单位。Auto代表是否是自动释放（其实不影响技能效果，主要是用于告诉其他人这个技能是不是自动释放的）。

ReadyChecker:技能除了基本判断（冷却，攻击次数，受伤次数）以外，其他的判断标准

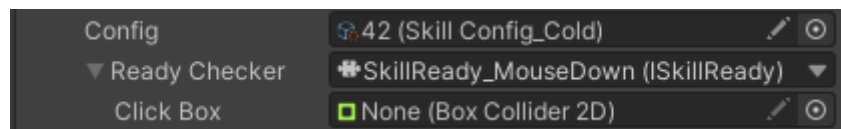


最常用的是SkillReady_IfTargetInRange：代表必须有目标在技能范围内才会使用技能，就不会冷却一到就使用技能



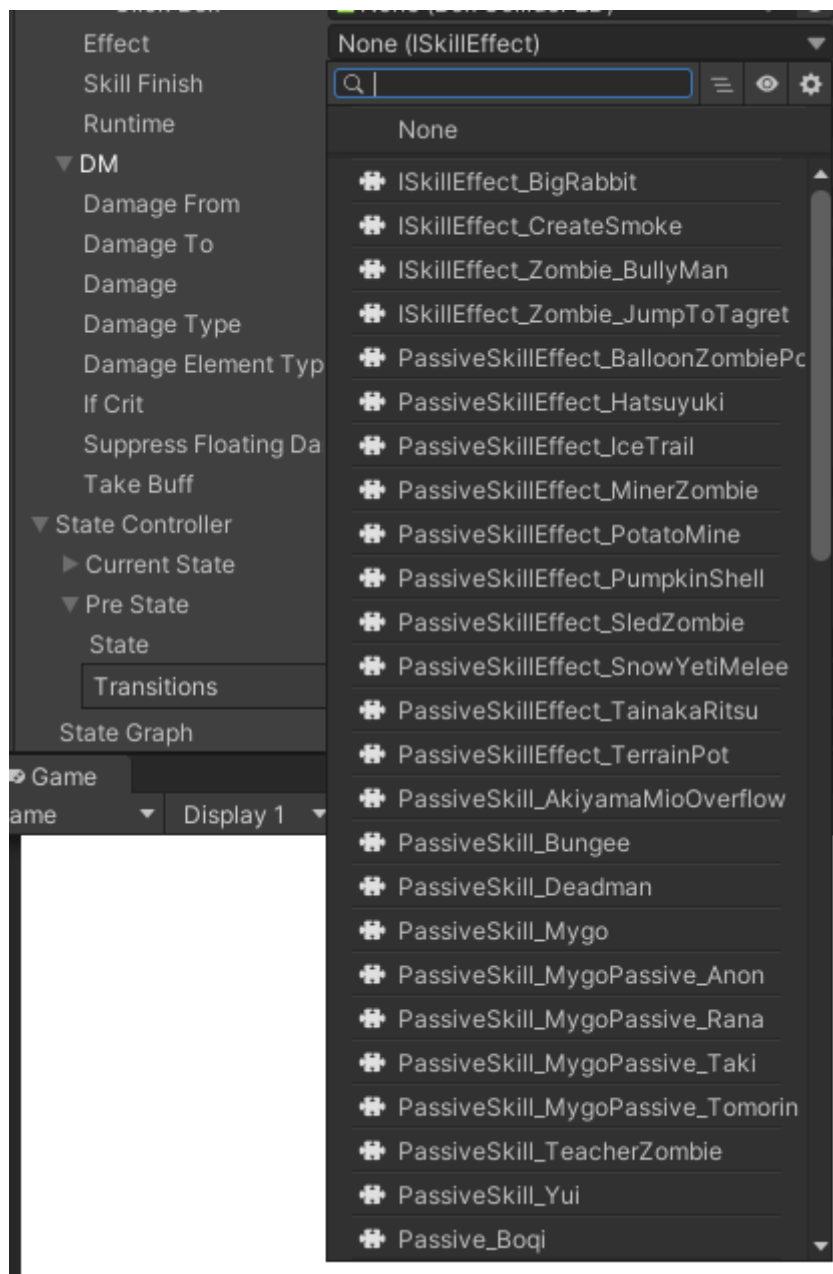
选中SkillReady_IfTargetInRange后再选择寻敌方式就可以了。这点和weapon的类似。

还有就是SkillReady_MouseDown：意思是必须鼠标点击后才会使用技能。



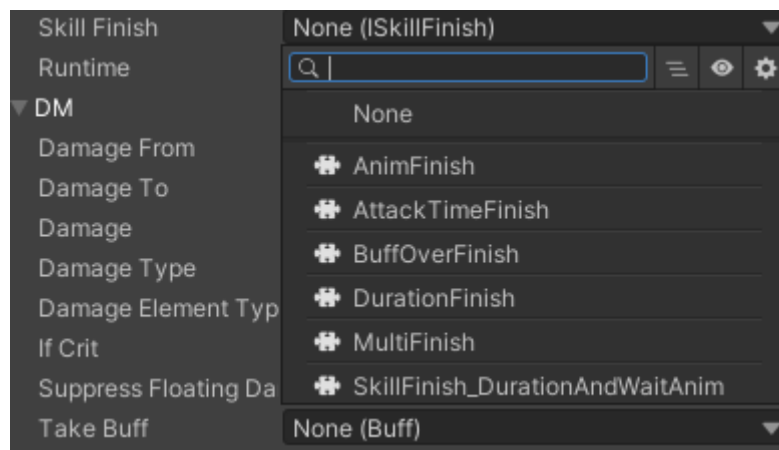
需要配置一个ClickBox 就是要一个检测点击范围的BoxCollider2D，我一般会放在Sprite子物体上 与chess身上的boxCollider做区别。

配置Effect:配置具体的技能效果：



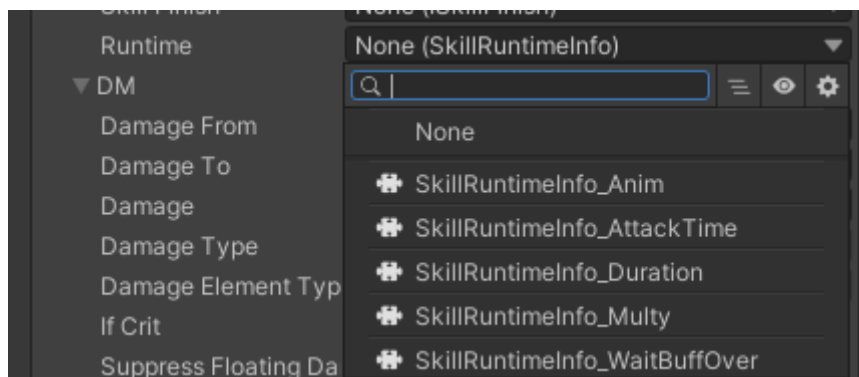
就是你技能的具体效果了，在角色播放技能动画到某一帧的时候会调用这个 具体的技能还要配置具体的信息。

skillFinish:技能的结束条件，就是什么情况算技能结束

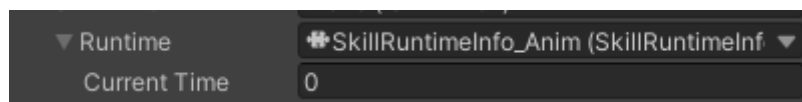


一般是AnimFinish 也就是技能动画播完就算结束；AttackTimeFinish使用几次技能后就算结束，一般是弹药类的技能使用；BuffOverFinish：技能效果是给自己施加一个Buff效果，这个技能会持续到buff结束；DurationFinish：设置一个持续时间，持续时间结束后算技能结束。

RunTime:与skillFinish配套使用的东西

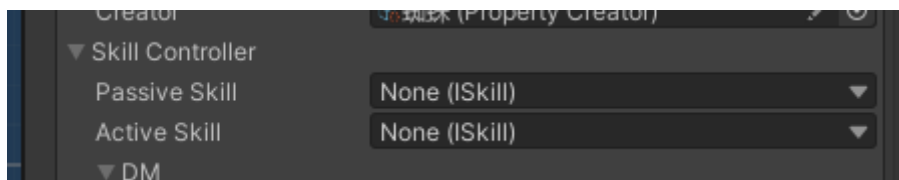


上面是怎么结束的 就在这里配置不同的数据，比如SkillRuntimeInfo_Anim。就直接设置就行了 没有其他数据需要配置。



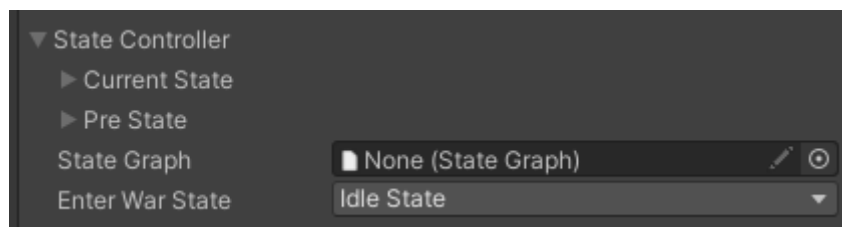
currenttime是内部数据不用改。

最后配置完的结果是这样的：

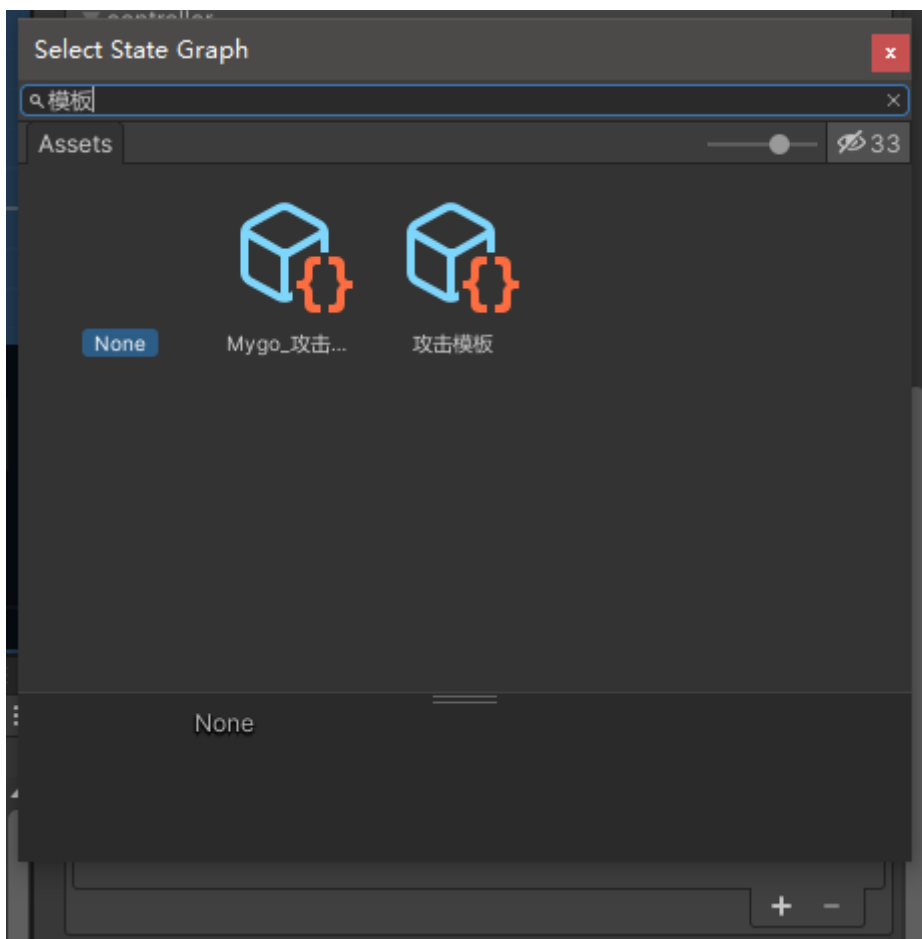


我们这个角色没有技能 所以直接空着就好

StateController:角色的状态机，因为我们这里面的所有角色都是用有限状态机配置的 所以如果有特殊情况就需要配置单独的StateGraph，一般情况下用模板就好。



这里面只有StateGraph和EnterWarState需要配置。后者代表你进入游戏的第一个状态是什么，一般情况下用idleState就可以。但比如说蘑菇他的基础状态可能是睡觉state,就需要修改一下。一般不改。然后是StateGraph,这个就比较复杂了



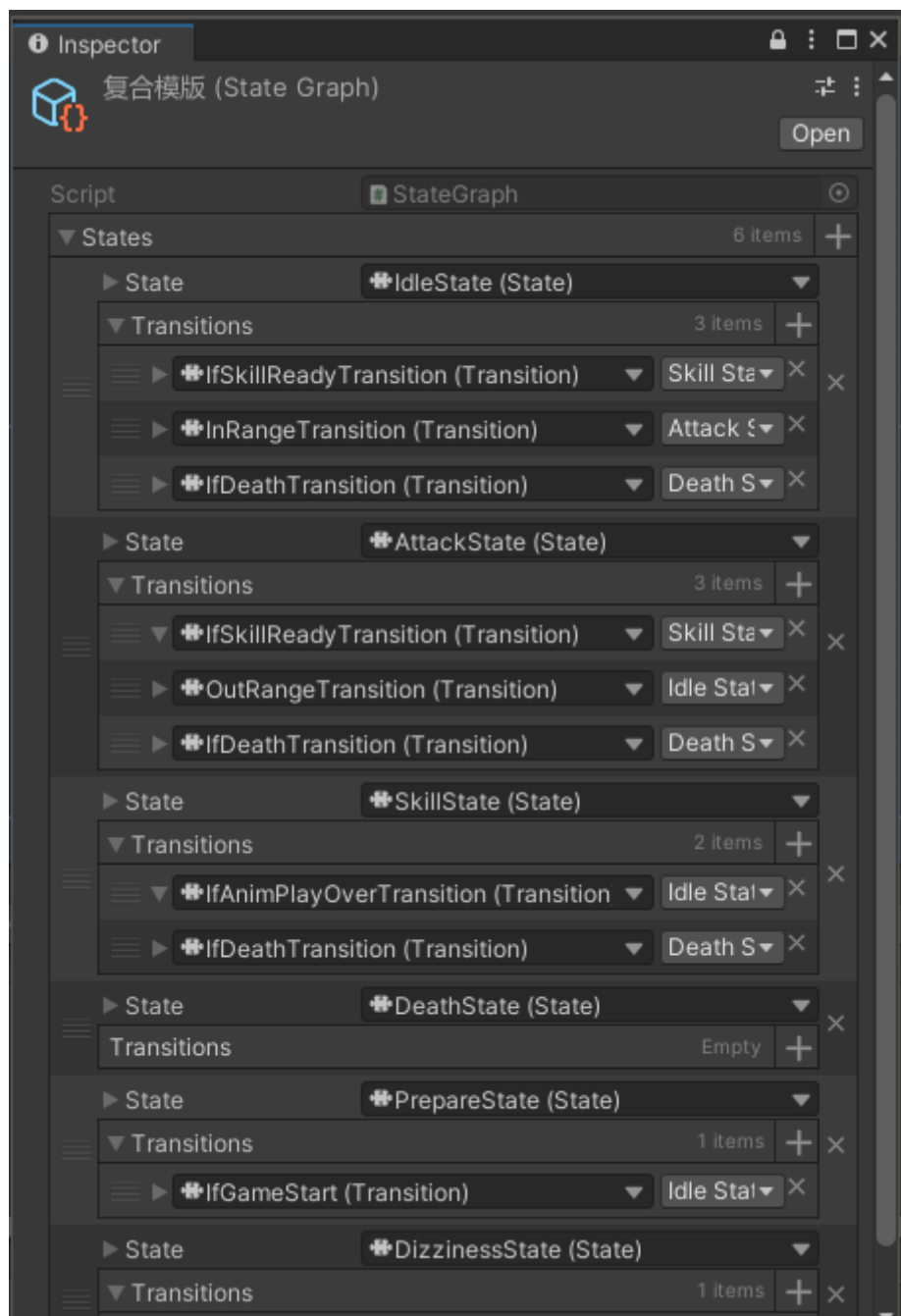
一般来说直接使用模板就可以了



僵尸模板就是通用的有移动 有攻击 没技能的模板；僵尸技能模板就是有技能的僵尸的模板 一般就着用着两个



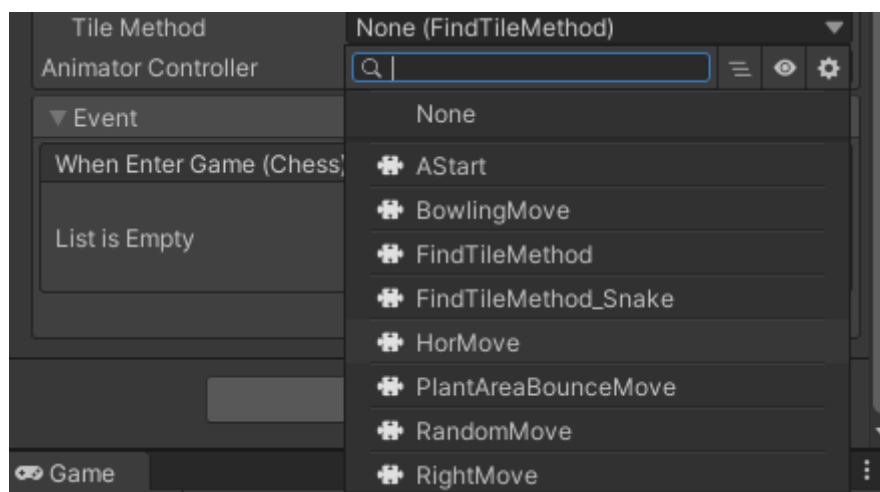
植物的话就是有复合模板：能攻击能用技能的模板（太多了）；技能模板：只能使用技能不能攻击的模板（向日葵）；攻击模板：只能攻击不能使用技能的模板（豌豆射手）。



至于具体的状态机配置 其实也不难 但我懒得写了 因为几百年也不会用到一个新的

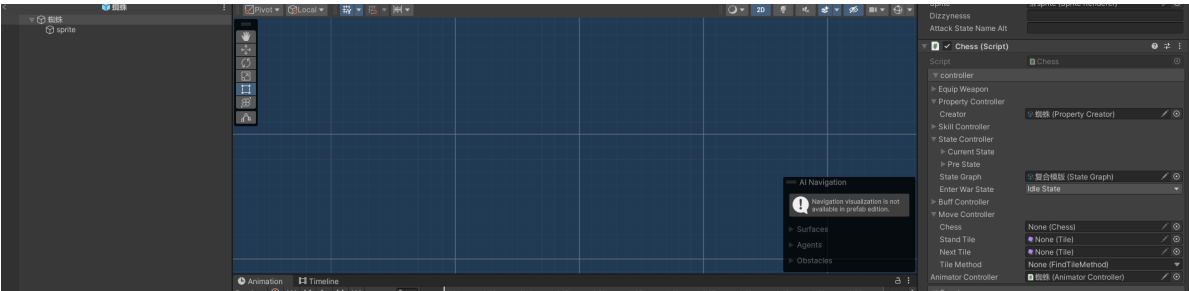
BuffController:不需要配置 都是内部逻辑

MoveController:僵尸的话需要配置，只需要修改TileMethor,选择HorMove就可以了



代表面向前方移动。至于其他的寻路方式 比如说BowlingMove就是坚果保龄球的移动方式，特定情况特定设置。一般用HorMove就可以了

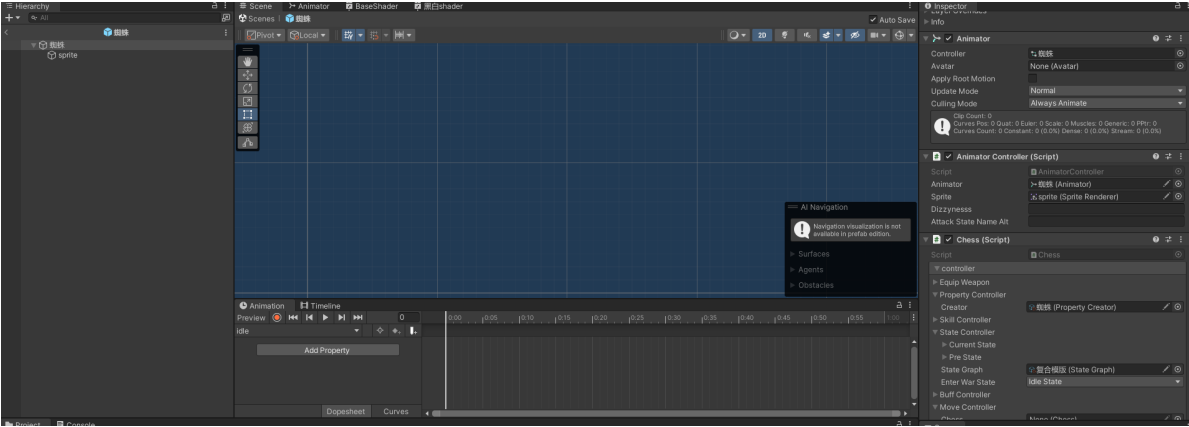
AnimatorController:动画控制器，直接把chess上挂载的AnimatorController拉进来就可以



直接把chess蜘蛛拉过来也是一样的

以上就是最关键的Chess的配置了。

AnimatorController的配置：



其实没什么好配置的，把挂载在Chess蜘蛛上的Animator组件拉到animator上，然后把sprite拉到Sprite上就可以了。（其实你创建的时候就已经拉好了）

以上就是制作一个植物的最最基础的步骤了

7. 技能与武器开发

7.1 文件放置

种类	目录示例
植物主动/效果	Assets/Script/Skill/ISkillEffect_Plant/
僵尸主动/效果	Assets/Script/Skill/ISkillEffect_Zombie/
被动	Assets/Script/Skill/ISkillPassive/ （可按子文件夹分 Zombie/Plant）
复合 Boss 技能	Assets/Script/Skill/skillBase/

新类用 [Serializable] 实现 ISkillEffect 或 ISkill，在 Prefab 的 SkillController 里 SerializeReference 选择。

7.2 常用集成 API

需求	API
生成棋子	<code>ChessTeamManage.Instance.CreateChess(creator, tile, tag)</code>
伤害	<code>DamageMessege</code> + 武器/技能内结算
无法选中	<code>Chess.UnSelectable()</code> / <code>ResumeSelectable()</code>
移动事件	<code>moveController.OnReachTile</code>
技能上下文	<code>skillController.context.Set(key, value)</code>
进场/离场	<code>WhenEnterGame</code> 、 <code>OnRemove</code> 、 <code>whenControllerLeaveWar</code>

复杂技能先写 `docs/requirements/*.md` 再编码（参考 `miner-zombie-passive.md`、`skill-effect-big-rabbit.md`）。

8. 注册到游戏

8.1 加入全局卡池 `GameManage.allChess`

1. 打开场景 `Assets/Scenes/开始.unity`（或挂有 `GameManage` 的入口场景）。
2. 在 `GameManage` 组件的 `allChess` 列表中追加本单位 `PropertyCreator`。

用途：图鉴、`Test` 模式、`PlantsShop` 兜底卡池、部分查找 `GetCreatorByChessName`。

8.2 植物：商店与选卡

接入	做法
植物商店 选卡	<code>PreParePlugun_ShowPlantShop.cards</code> 填入 <code>PropertyCreator</code> ；或依赖 <code>playerOwnedCreators</code>
玩家拥有	存档 <code>ownedCreatorIds</code> ； <code>AwardPanel.win()</code> / 金币商店 <code>ShopItemEffect_Creator</code>
种子 UI	配置 <code>PlantCardPre</code> 、 <code>PlantEntrepotCardPre</code>

8.3 僵尸：关卡出怪

1. 在 `Assets/Resources/LevelData/` 下打开目标 `LevelData` asset。
2. 配置 `zombieList` / 波次相关字段（与 `LevelController` 生成逻辑一致）。
3. 确认 `waveLimit`、`PlantType.LimitType` 等池化规则（`PropertyCreator.PassesWavePoolFilter`）。

测试面板：`TestScenePanel` 会从 `Resources/ChessData/Player` 与 `Enemy` 加载列表。

9. 测试清单

9.1 Test 模式快捷验证

- `GameManage.mode == Test`：启动时 `playerOwnedCreators = allChess`。
- 快捷键打开 `TestScenePanel` 生成植物/僵尸。
- 指定 `TestLevel` 进关验证波次与技能。

9.2 必测项

- ☐ `CreateChess` 后形象、层级、`tag` (`Player/Enemy`) 正确
- ☐ 植物：种植规则、`IfCanPlant`、重叠/Support 冲突
- ☐ 僵尸：出场格、移动、进家逻辑
- ☐ 普攻与技能 CD、动画事件与伤害一致
- ☐ 死亡：`DeathState` → 回收/事件解除，无泄漏
- ☐ 离场：`whenLeaveLevel` 后协程与监听已清理
- ☐ 存档（若可拥有）：`ownedCreatorIds` 与 `ApplyPlayerChessToGame` 后商店可见

9.3 性能

- 同屏多单位时技能勿每帧 `Instantiate`；用对象池（`CheckObjectPoolManage`）。
- Reanim 多部件注意 `Sorting Layer` / 合批。

10. 植物 vs 僵尸差异速查

步骤	植物	僵尸
PropertyCreator 路径	<code>ChessData/Player/</code>	<code>ChessData/Enemy/</code>
<code>ChessEditorWindow.enemy</code>	否	是
核心扩展	<code>IPlantFunction</code> 、 <code>PlantType</code>	<code>FindTileMethod</code> 、波次限制
典型接入	<code>PlantsShop</code> 、关卡 <code>cards</code> 、奖励	<code>LevelData</code> 波次、Boss 召唤池
图层/碰撞	与现植物一致	与现僵尸一致

11. 常见问题

现象	排查
关卡刷不出	<code>LevelData</code> 是否引用； <code>waveLimit</code> / 池过滤
商店没有	<code>allChess</code> / <code>playerOwnedCreators</code> / <code>PreParePlugun_ShowPlantShop.cards</code>
技能不触发	动画是否调用 <code>UseSkill</code> ； <code>IsSkill</code> 是否 <code>InitSkill</code>
种植失败	<code>plantFunction</code> 、 <code>chessTileType</code> 与地图格子类型

现象 creator 为 null	排查 chessName 与 allChess 不一致
----------------------	--------------------------------

12. 文档与审批（复杂单位）

产出	路径
需求卡片	docs/requirements/[单位名].md
Reanim 资源索引	docs/Reanim2UnityAnim-inventory.md
架构/影响面	由 @architecture-design-analyzer、@impact-scope-analyzer 生成

13. 最小步骤摘要（熟练后）

- 复制同类 Prefab + PropertyCreator → 改 chessName、数值、贴图。
- 改/写 iskillEffect 与 weapon，挂到 Prefab。
- 补动画与 Animation Event。
- PropertyCreator.chessPre 指向 Prefab；allChess 注册。
- 关卡或商店接入 → Test 场景验证 → 需要时补需求文档。

文档版本：2026-05-22 · 与 context/ 及 AGENTS.md 约定一致；架构变更请同步更新 context/modules/Chess.md。